

KAMP 제조AI데이터셋 활용 권한

No. GB20250818-PU1755498594-243778



⚠ 제조AI데이터셋 활용 주의사항

연구/공식적으로 사용하실 때에는 꼭 아래와 같이 'KAMP 출처(reference)'를 남겨주시고, 인용 시 활용한 내용과 문서 등은 아래 이메일로 보내주시기를 바랍니다.
(E-mail : kamp@kaist.ac.kr)

국문 출처 표기 양식

중소벤처기업부, Korea AI Manufacturing Platform(KAMP), 표면처리 공급망 최적화 AI 데이터셋, KAIST(울산과학기술원, (주)에이비에이치), 2022.12.23., www.kamp-ai.kr

영문 출처 표기 양식

Ministry of SMEs and Startups., and KAIST (Korea Advanced Institute of Science and Technology). (2022, December 23). Surface treatment supply chain optimization AI dataset. Korea AI Manufacturing Platform(KAMP). <https://www.kamp-ai.kr/>

제조AI데이터셋 개요

제조AI데이터셋명 : 표면처리 공급망 최적화 AI 데이터셋

업종 : 뿌리(표면처리) | **목적** : 공급망 최적화 | **유형** : csv

알고리즘 : 유전 알고리즘(Genetic Algorithm)

사용조건 : 콘텐츠 변경 허용

제공기관 : KAIST (수행기관 : 울산과학기술원/(주)에이비에이치)

등록일 : 2022-12-23

이용자 정보

회원 ID : 01soju

다운로드일 : 2025-08-18 17:20:58

활용목적 : 공부

인공지능 제조 플랫폼(KAMP, Korea AI Manufacturing Platform)에서 다운로드한 제조AI데이터셋의 저작권을 위하여 문서 고유 번호와 워터마크를 부여하여, 해당 문서의 표준을 준수하였음을 증명합니다.

운영기관 : KAIST 제조AI빅데이터센터

주소 : 대전광역시 유성구 문지로 193 KAIST 문지캠퍼스 행정동

전화번호 : 042-350-1331 | 이메일 : kamp@kaist.ac.kr

「표면처리 공급망최적화 AI 데이터셋」

분석실습 가이드북



「표면처리 공급망최적화 AI 데이터셋」

분석실습 가이드북



Contents

1 분석요약	04
2 분석 실습	
1. 분석 개요	05
1.1 분석 배경	05
1) 공정(설비) 개요	
2) 이슈사항(Pain point)	
1.2 분석 목표	09
1) 분석 목표	
2) 데이터 정의 및 소개	
3) 데이터 분석 기대효과	
4) 시사점(implication) 요약기술	
2. 분석실습	10
2.1 제조데이터 소개	10
1) 데이터 수집 방법	
2) 데이터 유형/구조	
3) 데이터 (품질) 전처리	
2.2 분석 모델 소개	17
1) 데이터 흐름 및 인공지능 모델 적용 흐름도	
2) AI 분석모델	
2.3 분석 체험	22
1) 필요 SW, 패키지 설치 방법 및 절차 가이드	
2) 분석 단계별 프로세스 - Flow Chart	
[단계 ①] 라이브러리/데이터 설치 및 불러오기	
[단계 ②] 데이터 전처리	
[단계 ③] 유전 알고리즘 구축 및 실행	
[단계 ④] 결과 시각화 및 도출	
[단계 ⑤] 결과 분석 및 해석	
3. 유사 타현장의 분석 적용	40
부 록	
분석환경 구축을 위한 설치 가이드	41
데이터 품질 전처리 [실습 코드]	54



「표면처리 공급망최적화 AI 데이터셋」 분석실습 가이드북

- 필 요 S W : Python, Anaconda - Jupyter Notebook
- 필요 패키지 : Pandas, Numpy, Datetime, Matplotlib, PyGAD
- 분석 환경 : [운영체제] Window, Ubuntu 14.0 이상, [CPU] AMD Razer 3700x 2.3 GHz, [Ram] 16GB
- 필요 데이터 : 일무사_표면처리.csv

1 분석요약

No	구 분		내 용
1	분석 목적 (현장 이슈, 목적)		제조업은 물품을 수주 및 발주하여 공정을 진행하는 특성상 다양한 기업과 네트워크로 이어져 있다. 기업의 상황에 맞는 원자재 수급을 위한 공정 및 대치가 유연해야 한다. 하지만 여전히 많은 기업에서 내부 업체의 데이터 관리가 원활하게 이루어지지 않고 있다. 그로 인해 특이사항이 발생하는 경우 유연한 대처에 어려움을 겪는다. 따라서 수주, 발주 및 원자재 공급을 할 수 있도록 정보가 제공되고 이에 맞는 최적의 공정 계획을 세울 수 있는 가이드 시스템이 필요하다.
2	데이터셋 형태 및 수집방법		1) 분석에 사용된 변수 : 생산일자, 작업지시번호, 제품코드, 시작, 종료, 생산량, 1~10차 출고일, 1~10차 출고량 2) 데이터 수집 방법 : 원자재 및 중간재, 완제품 유통 데이터 수집 장비를 통해 제품 생산 주문 리스트 도착 시 하루 2회 수집 3) 데이터셋 파일 확장자 : CSV
3	데이터 개수 데이터셋 총량		- 데이터 개수 : Row 수 (177개), Column 수 (26개), 총 3,487개 - 데이터셋 총량 : 62 KB
4	분석적용 알고리즘	알고리즘	유전 알고리즘(Genetic Algorithm)
		알고리즘 간략소개	유전 알고리즘은 변이과정과 적자생존으로 최적의 개체를 유전하는 자연현상을 최적화에 적용한 알고리즘이다. 해당 알고리즘은 임의의 조합해 집합을 생성, 최적도가 높은 조합해를 선택, 재조합, 변이하는 일련의 과정을 반복하며 더 나은 조합해를 찾아가는 과정이다. 이 알고리즘의 장점은 목적함수의 수치해석을 이용하지 않기 때문에, 수치 해석적 최적 해를 찾기 힘든 목적함수나 제약조건을 가지는 문제의 솔루션 도출에 효과적이다.
5	분석결과 및 시사점		- 원자재 및 완제품의 전반적인 공급 유통망 구조를 포함한 공정 데이터셋을 분석함으로써 유통 구조에 위험요소가 될 수 있는 특이사항을 검출하고 조기 대응하는 전략을 세우고자 한다. - 이를 기반으로 복잡한 유통 구조에 맞춤형 공정 계획을 최적화한다.

2 분석 실습

1. 분석 개요

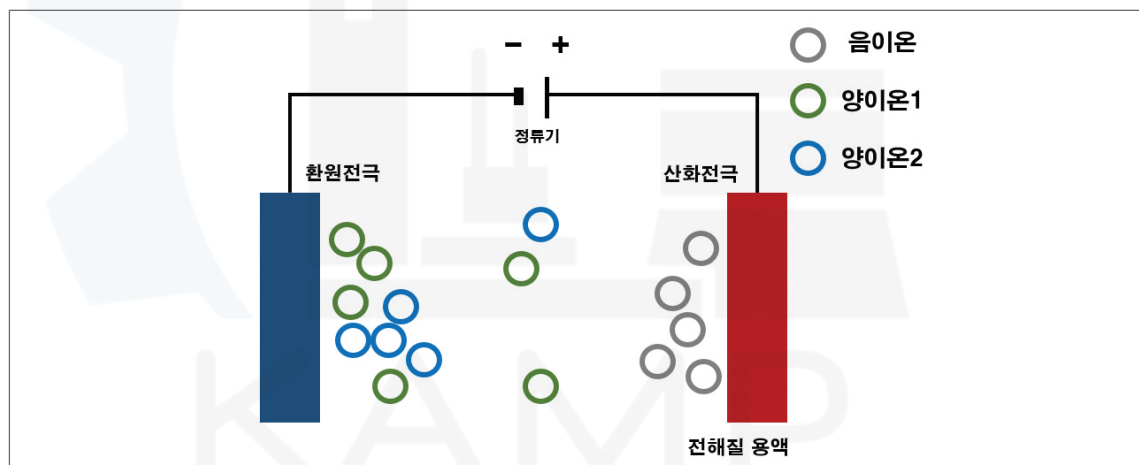
1.1 분석 배경

1) 공정(설비) 개요

- 현재 해당 기업은 볼트와 너트 및 철판에 대한 전기 아연도금을 수주하는 회사로서 다양한 기업과 연계하여 물품을 수주 및 발주하며 공정을 진행하고 있다.
- 표면처리 산업은 금속 소재 가공품을 후방산업으로 하여 제조 서비스업 분야 내 복잡하게 연결되는 특성을 가지므로 이에 공급망 형태가 단일 방향이 아닌 방사형의 복잡한 네트워크를 가지게 된다.

• 전기아연도금 공정

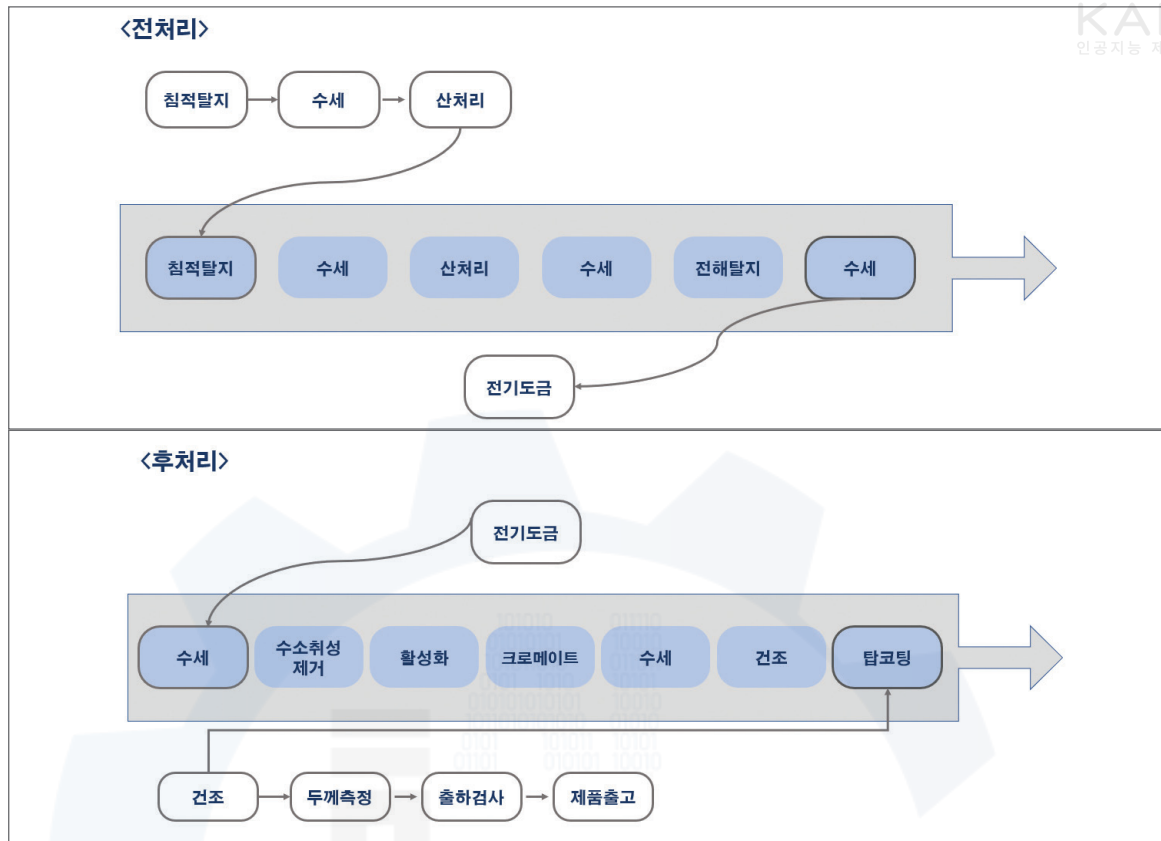
- 표면처리 공정의 일종으로 전기아연도금 공정은 잔류가 통하는 금속 모재를 도금액에 침적시켜 전류를 통과 및 모재 표면에 얇은 아연 피막층을 형성시키는 공정이다.



[그림 1] 도금 공정 개요 1

- 도금 공정은 연속공정으로서 자동화 설비에 의해 표면 세척부터 도금 후처리에 이르는 과정이 진행된다.
- 이후 품질검사를 거쳐 열풍건조 과정에 따라 공정은 완료된다.
- 부품 및 소재류의 물리화학적 특성을 강화하는 특징에 따라 공정의 대상이 되는 제품 종류가 약 100여가지 이상에 이른다.
- 이에 따라 제품 종류별 공정 조건의 변동이 발생한다. 이 과정에서 작업자의 노하우에 의존하며 공정변수 선정이 진행된다.

• 전기아연도금 설비 및 기업공정 특성



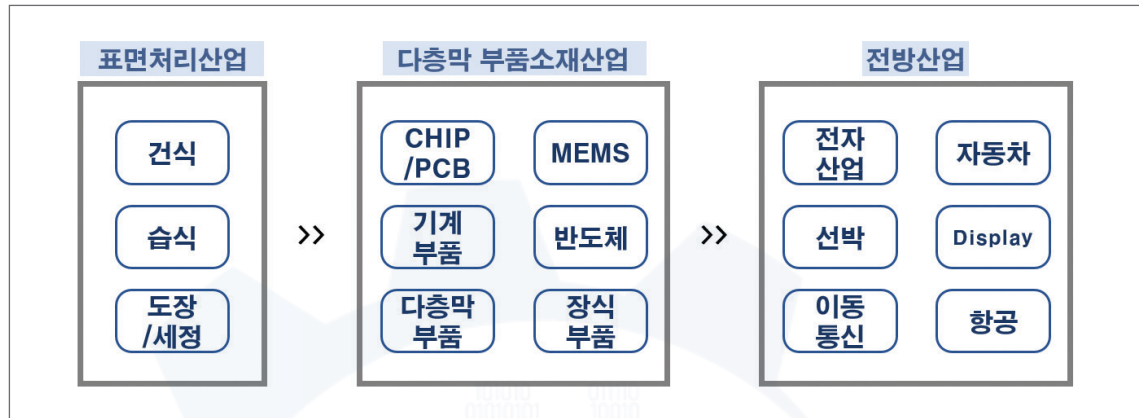
[그림 2] 도금 공정 개요 2

- 현재 해당 기업은 전기아연도금을 기반으로 기업 특유의 기술을 적용한 자동화 설비를 활용하여 해당 과정에서 데이터가 자동적으로 수집된다.
- [그림 2]에서 볼 수 있듯이, 첫 단계로 도금체 표면에 묻은 이물질을 제거하고자 산성 물질에 담궈 수세 작업을 진행한다.
- 1차 수세 작업 마무리 후 기포를 활용한 미세 이물질과 기름때를 지우는 전해탈지가 진행된다. 이후 전기아연도금을 진행한다.
- 전기아연 도금이 마무리되면 아연도금 표면의 부식을 늦추기 위한 크로메이트 코팅이 진행된다.
- 마지막으로 코팅된 부분을 건조하여 최종 화학 결합을 통해 표면을 안정시키는 작업을 진행한다.

• ECR 방식 공급망 관리 특성

- ECR (Efficient Consumer Response)이란 효율적인 소비자 대응을 위하여 유통업체와 제조업체가 서로 밀접하게 데이터를 공유하여 수요에 대한 예측을 진행하고 공정을 진행하는 전략이다.

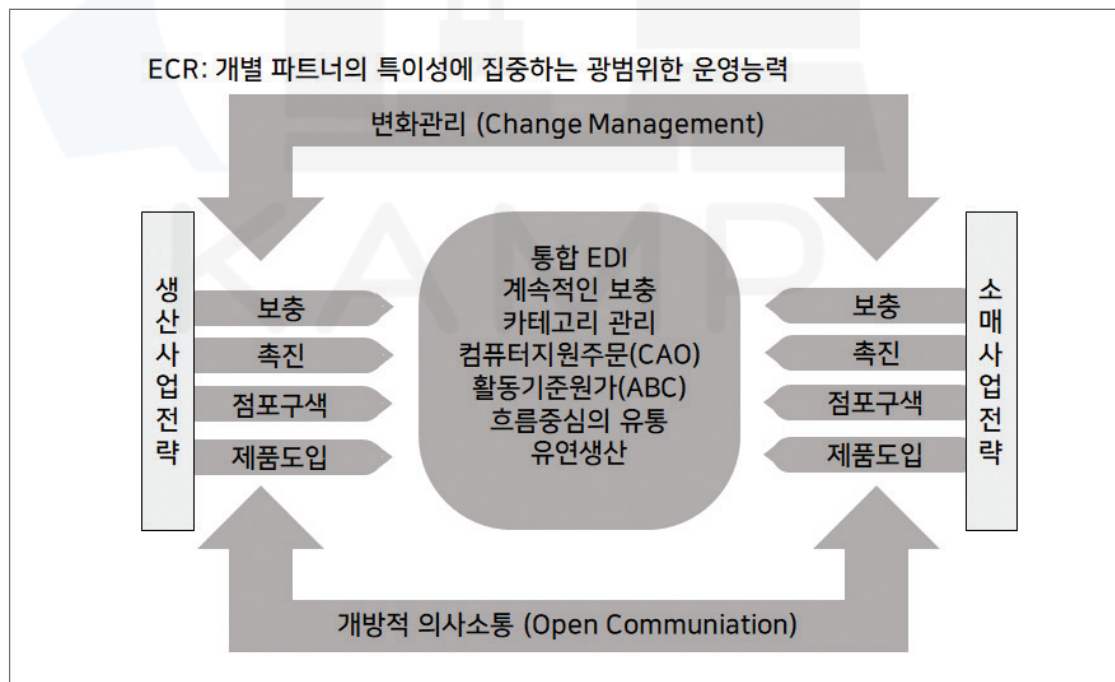
- 현재 해당 기업은 연계된 타기업들과 전기아연도금 유통·공급망을 형성하여 ECR 방식을 활용한 공급망 관리를 진행하고 있다.
- 특히 제조업체, 유통업체, 하청업체 등의 이해관계자들이 서로 협력하여 소비자의 요구에 효율적이고 빠르게 대응하기 위한 전략으로서 최근 다양한 산업에 전반적으로 확산되고 있다.



[그림 3] 표면처리 산업구조

- ECR은 다음의 산업군에서 활용할 때 효율적이다.

- ① 제품이 중간 유통망을 거치는 업종
- ② 데이터 교환이 가능한 시스템이 구축된 업체
- ③ 식품 제조업



[그림 4] ECR work flow

- ECR 장점 : 유통망의 수요에 따르는 생산량 예측이 가능하며 이에 따라 제공 재고가 감소하여 기회비용이 절감하고 공급 속도가 유연하게 된다. 또한, 이에 따라 재정 비용 감소까지 가능하다.
- ECR 단점 : 유통망의 데이터를 기반으로 진행되기 때문에 유통망 수요 데이터가 정확하지 않으면 수요 예측의 정확성에 문제가 생기는 이슈가 있다.

2) 이슈사항(Pain point)

• 공정(설비)상의 문제 현황

- 제조 공정에서는 다양한 이슈로 문제들이 발생하기 때문에 이를 올바르게 해결하기 위해서는 현상에 대한 이해가 필요하다. 문제에 대한 원인은 한 가지 혹은 동시에 얹혀서 나오기 때문에 문제 상황에 대한 대책을 세울 때 원인과 현상에 대한 이해는 반드시 필요한 부분이다. 따라서 해당 공정 관련 이슈 사항들을 다음과 같이 나열했다.
- 현재 수주 및 발주 중심으로 생산이 진행되는 제조업의 특성상 공급망 내 기업의 상황에 따라 수주·발주량은 민감하게 반응하며 끊임없이 변동한다. 따라서 수주·발주처의 상황 변동에 대한 유연한 대처 방안과 공정 원자재 수급의 탄력성을 높일 수 있는 대책이 필요한 상황이다.
- 특히 해당 업체의 경우는 수주를 받고 공정을 진행하기 위해 사용하는 원자재까지 수주량을 예측하여 재고를 쌓아두고 있다.
- 도금을 위해 필요한 도금용재부터 수세에 활용하는 산성용액, 크로메이트 용액까지 공정을 위해 필요한 원자재가 다양한데 이 원자재를 각기 다른 업체로부터 구매하여 재고를 보유하고 있는 상황이다.
- 현재는 수주량에 대한 예측이 원활하지 않아 재고가 부족하여 납기일이 지연되는 경우도 발생하고, 과하게 구매함으로써 원자재 가격 변동에 피해를 보는 경우도 존재한다.

• 문제해결 장애요인

- 현재 네트워크를 형성하고 있는 업체들에 대한 데이터 분석이 원활하게 진행되지 않아 잠재적 위험 요소의 도출 및 특이사항에 대한 대처 방안이 확실하게 마련되지 않은 상황이다.
- 해당 기업의 경우, 공정은 RtoR 방식으로 진행하므로 제품의 정확한 개수와 품질 여부 체크가 힘들다. 현재는 생산품의 무게를 측정하여 재고를 파악하고 있는데 품질에 대한 정보가 정확하지 않아 정상 품질의 재고가 얼마큼 있는지, 판매 불가능한 제품이 있는지 확인하는 과정에 어려움이 있다. 따라서 상품성이 확실한 제품의 재고 및 생산량 예측에 어려움을 겪고 있다.



- 이에 따라, 안정적인 원자재의 재고 확보를 위한 수주량 예측 정확도가 떨어질 수밖에 없는 상황이다.

• 극복 방안

- AI 알고리즘을 활용하여 데이터 분석을 진행하고 특이사항이 발생할 경우 유연한 수주·발주 및 원자재 공급을 할 수 있도록 정보를 제공하고자 한다. 특히 공정계획을 세울 때 도움이 되는 가이드 시스템을 구축하고자 한다.
- 1차적으로 RtoR 방식을 통해 제조하는 볼트 및 너트 제품에 대한 수주량을 예측하게 된다면 단순 계산으로 약 95%(불량률 최대 기준) 이상에 해당하는 원자재를 예측하여 확보가 가능한 것을 본 가이드북 분석을 통해 확인한다. 이를 기준으로 여분을 남겨 추가적인 원자재를 확보한다면 기존에 발생하던 문제인 원자재 부족으로 인한 납기일 지연을 해결할 수 있을 것으로 예상된다.
- 추후 추가 제품에 대한 데이터를 수집하고 이를 활용한 솔루션 확장을 통해 원자재 가격 변동으로 인한 금액 손해도 해결할 수 있을 것으로 예상된다.

1.2 분석 목표

1) 분석 목표

- 현재 원자재 및 중간재 수급부터 완제품 공급 및 유통망 구조까지 포함한 데이터셋에 대하여, AI 알고리즘을 활용한 데이터 분석을 통해 탄력적 방안을 모색하고자 한다.
- 메인 제품인 볼트와 너트에 대한 수주량(혹은 판매량) 예측을 진행하여, 공정에 필요한 다양한 원자재 사용량을 예측할 수 있도록 분석 데이터를 제공하는 것이 목표이다. 이를 활용하여 원자재 부족으로 인한 납기일 지연 문제까지 해결하고자 한다.

2) 데이터 정의 및 소개

- 해당 분석에 사용되는 공정 데이터들은 실제 2021년 1월부터 2022년 5월까지 유통 및 공정 데이터, 생산데이터(제품코드, 생산량, 출고일자, 출고량)를 활용한다.

3) 데이터 분석 기대효과

- 데이터 분석을 통해 유통 구조에 위험요소가 될 수 있는 원인을 파악하여 조기대응할 수 있는 전략을 세울 수 있다. 공급망의 안정성 확보에 기여할 수 있을 것으로 기대한다.
- 복잡한 유통 구조에 대한 맞춤형 솔루션을 제공함으로써 수주 발주 및 원자재 공급을 할 수 있도록 적절한 정보를 제공하고 이를 기반한 공정 계획을 세울 수 있을 것으로 기대한다.



4) 시사점(implication) 요약기술

- 현재 해당 공정에서는 수주를 받을 때 제품 생산이 들어가면서 정해진 날짜에 출고를 하게 되는 상황이다. 원본 데이터상 생산 일자가 모두 다르므로 생산 시작 날짜를 고려한 생산량을 예측할 수 있는 AI 기법의 적용이 필요한 상황이다. 이에 유전 알고리즘 (Genetic Algorithm)을 모델 적용을 통해 출고일에 맞춘 최적의 생산량을 예측할 수 있는 분석을 수행하고자 한다.
- 본 가이드북에서는 제조업에서 남은 재고량을 최소화하기 위해 기존 원본 데이터에 존재하지 않는 재고 패널티를 추가로 고려한다. 이는 일일 최대 생산량을 하이퍼파라미터로 조정하며 네 가지 시나리오를 제시한다. 제시된 시나리오들 중 최고의 솔루션과 base solution 값의 차가 큰 파라미터 조합을 선택하거나, 일일 최대 생산량과 같은 조정을 통해 타 제조업 맞춤 커스터마이징을 할 수 있는 솔루션을 제안한다는 점에서 시사점이 크다고 판단된다.

2. 분석실습

2.1 제조데이터 소개

1) 데이터 수집 방법

- 제조 분야 : 사출성형
- 제조 공정명 : 표면처리 공정
- 수집장비 : 원자재 및 중간재, 완제품 유통 데이터
- 수집 기간 : 2021/01/12 ~ 2022/05/03 (약 1년 4개월)
- 수집 주기 : 제품 생산 주문 리스트 도착할 시 수집(하루 2회)

2) 데이터 유형/구조

- 데이터셋 구조 : 테이블 형식(tabular)
- 데이터 개수 : Row 177개, Column 26개, 총 3,487개
- 제조AI데이터셋 주요 변수 정의



① 원본 데이터 : 일무사_표면처리.csv

생산일자	작업지시번호	제품코드	시작	종료	생산량(Kg)				
2021-01-12	2101130065	M21	2021-01-13 7:10	2021-01-13 10:40	13,550				
2021-01-12	2101130066	M21	2021-01-13 10:40	2021-01-13 14:05	14,000				
2021-01-13	2101130067	M21	2021-01-13 14:05	2021-01-13 17:30	14,000				
2021-01-13	2101130068	M21	2021-01-13 17:30	2021-01-13 20:40	14,000				
2021-01-13	2101140075	M21	2021-01-13 20:40	2021-01-13 23:50	15,500				
2021-01-18	2101190091	M21	2021-01-19 4:40	2021-01-19 8:00	14,000				
2021-01-18	2101190092	M21	2021-01-19 8:00	2021-01-19 11:26	14,000				
2021-01-18	2101190093	M21	2021-01-19 11:25	2021-01-19 14:55	14,000				
2021-01-19	2101190094	M21	2021-01-19 14:55	2021-01-19 18:30	16,000				
2021-01-27	2101270149	M21	2021-01-27 17:30	2021-01-27 21:15	14,000				
2021-01-27	2101270150	M21	2021-01-27 21:15	2021-01-28 0:45	14,000				
2021-01-27	2101280153	M21	2021-01-28 0:45	2021-01-28 4:15	14,000				
2021-01-27	2101280154	M21	2021-01-28 4:15	2021-01-28 7:45	14,000				
2021-01-27	2101280155	M21	2021-01-28 7:45	2021-01-28 11:10	14,500				
2021-02-09	2102100254	M21	2021-02-10 8:45	2021-02-10 11:55	14,000				
2021-02-09	2102100255	M21	2021-02-10 11:55	2021-02-10 15:15	14,000				
2021-02-10	2102100256	M21	2021-02-10 15:15	2021-02-10 18:35	13,450				
2021-02-22	2102230316	M21	2021-02-23 2:25	2021-02-23 5:50	14,000				
2021-02-22	2102230317	M21	2021-02-23 5:50	2021-02-23 9:25	13,550				
2021-02-22	2102230318	M21	2021-02-23 9:25	2021-02-23 13:00	14,000				
2021-02-23	2102230319	M21	2021-02-23 13:00	2021-02-23 16:50	15,400				
2021-03-02	2103020362	M21	2021-03-02 16:45	2021-03-02 20:05	14,000				
2021-03-02	2103020363	M21	2021-03-02 20:05	2021-03-02 23:25	13,600				
2021-03-02	2103020364	M21	2021-03-02 23:25	2021-03-03 2:50	14,000				
2021-03-02	2103020365	M21	2021-03-03 2:50	2021-03-03 6:10	14,000				
2021-03-02	2103030366	M21	2021-03-03 6:10	2021-03-03 9:40	14,500				
2021-03-16	2103170471	M21	2021-03-17 7:50	2021-03-17 11:05	13,500				
1차 출고날짜	2차 출고날짜	3차 출고날짜	4차 출고날짜	5차 출고날짜	6차 출고날짜	7차 출고날짜	8차 출고날짜	9차 출고날짜	10차 출고날짜
2021-01-25	2021-01-26	2021-01-19	2021-01-28						
2021-01-19	2021-01-25								
2021-01-28	2021-02-09	2021-02-17	2021-02-03						
2021-01-26	2021-01-25								
2021-02-17	2021-02-25	2021-02-22	2021-03-12	2021-03-04					
2021-02-03	2021-02-18	2021-02-17	2021-01-26	2021-02-04	2021-02-09				
2021-02-03	2021-02-05	2021-02-04							
2021-02-03	2021-01-26								
2021-03-08	2021-03-24	2021-02-03	2021-03-12	2021-03-19	2021-03-04				
2021-02-22	2021-03-24	2021-03-30							
2021-02-05	2021-02-25	2021-02-08	2021-03-04						
2021-02-18	2021-02-22	2021-02-17							
2021-02-10	2021-02-08	2021-02-09							
2021-02-23	2021-02-17	2021-02-22	2021-02-10	2021-03-03					
2021-02-23	2021-02-22	2021-03-10							
2021-02-26	2021-02-23								
2021-02-26	2021-02-22								
2021-03-19	2021-03-30	2021-04-07							
2021-03-03	2021-03-10	2021-03-18							
2021-03-12	2021-03-26	2021-02-26	2021-03-19	2021-03-25	2021-03-03	2021-03-24			
2021-03-05	2021-03-18	2021-03-10							
2021-04-12	2021-04-16	2021-04-07	2021-03-30	2021-04-08					
2021-03-18	2021-03-22	2021-03-24							
2021-03-18									
2021-03-25	2021-03-24								
2021-03-26	2021-03-30	2021-03-25							
2021-04-01	2021-03-26	2021-04-05	2021-04-06	2021-03-30					

1차 출고량	2차 출고량	3차 출고량	4차 출고량	5차 출고량	6차 출고량	7차 출고량	8차 출고량	9차 출고량	10차 출고량
2,000	1,000	9,550	1,000	0	0	0	0	0	0
4,000	10,000	0	0	0	0	0	0	0	0
1,000	6,000	6,000	1,000	0	0	0	0	0	0
12,000	2,000	0	0	0	0	0	0	0	0
4,000	5,000	4,000	500	2,000	0	0	0	0	0
3,000	1,000	1,000	4,000	4,000	1,000	0	0	0	0
5,000	4,000	5,000	0	0	0	0	0	0	0
4,000	10,000	0	0	0	0	0	0	0	0
5,000	500	2,000	3,500	1,000	4,000	0	0	0	0
1,000	12,500	1,000	0	0	0	0	0	0	0
1,000	1,000	11,000	1,000	0	0	0	0	0	0
4,000	1,000	9,000	0	0	0	0	0	0	0
2,000	6,000	6,000	0	0	0	0	0	0	0
2,000	3,000	1,000	7,000	1,000	0	0	0	0	0
9,000	1,000	4,000	0	0	0	0	0	0	0
7,000	7,000	0	0	0	0	0	0	0	0
450	13,000	0	0	0	0	0	0	0	0
6,000	4,000	4,000	0	0	0	0	0	0	0
7,000	4,000	3,000	0	0	0	0	0	0	0
1,000	2,000	550	1,000	1,000	6,000	2,000	0	0	0
8,000	400	7,000	0	0	0	0	0	0	0
5,000	3,000	4,000	1,000	1,000	0	0	0	0	0
2,000	6,000	6,000	0	0	0	0	0	0	0
13,600	0	0	0	0	0	0	0	0	0
4,000	10,000	0	0	0	0	0	0	0	0
8,000	1,500	5,000	0	0	0	0	0	0	0
3,350	1,000	2,000	8,000	1,000	0	0	0	0	0

[그림 5] 비식별화된 원본 데이터(일무사_표면처리.csv) 예시

- 현재 제공된 원본 데이터는 표면처리 공정 과정에서 수집된 생산 리스트 원본 데이터 셋으로 csv(텍스트 파일)이 [그림 5]와 같은 형태로 제공된다.

• 원본 데이터(일무사_표면처리.csv) 속성정의 표

Index	Attributes name	Description	Data Type	Unit
1	생산일자	생산일자	object	177
2	작업지시번호	작업지시 번호	float	177
3	제품코드	제품의 종류	object	177
4	시작	생산 시작 시간	object	177
5	종료	생산 종료 시간	object	177
6	생산량	제품이 생산된 수량	object	177
7	출고날짜	1~10차 제품 출고일	object	177
8	1~10차 출고량	1~10차 제품 출고량	object	177



② 가공 데이터: Base Solution 도출

	date	produce	release	19	2021-01-31	0	0
0	2021-01-12	0	0	20	2021-02-01	0	0
1	2021-01-13	71050	0	21	2021-02-02	0	0
2	2021-01-14	0	0	22	2021-02-03	0	15000
3	2021-01-15	0	0	23	2021-02-04	0	9000
4	2021-01-16	0	0	24	2021-02-05	0	5000
5	2021-01-17	0	0	25	2021-02-06	0	0
6	2021-01-18	0	0	26	2021-02-07	0	0
7	2021-01-19	58000	13550	27	2021-02-08	0	17000
8	2021-01-20	0	0	28	2021-02-09	0	13000
9	2021-01-21	0	0	29	2021-02-10	41450	9000
10	2021-01-22	0	0	30	2021-02-11	0	0
11	2021-01-23	0	0	31	2021-02-12	0	0
12	2021-01-24	0	0	32	2021-02-13	0	0
13	2021-01-25	0	14000	33	2021-02-14	0	0
14	2021-01-26	0	27000	34	2021-02-15	0	0
15	2021-01-27	28000	0	35	2021-02-16	0	0
16	2021-01-28	42500	2000	36	2021-02-17	0	23000
17	2021-01-29	0	0	37	2021-02-18	0	5000
18	2021-01-30	0	0	38	2021-02-19	0	0

[그림 6] 가공 데이터 (Base_Solution.csv) 예시

- 기존 원본 데이터 (일무사_표면처리.csv)의 경우 26개 column의 데이터가 있으며, 수량 예측을 위해 '생산량(kg)' 변수만을 고려했다.
- '생산일자' column을 기준으로 지정하여 오름차순으로 데이터를 정리한 후 생산일자의 최솟값 및 최댓값을 계산한다.
- 계산된 해당 날짜마다 생산량과 출고량을 이용하여 Date, Produce, Release 형태의 새로운 dataframe을 생성한다.
- 따라서 최종 가공 데이터(Base_Solution.csv)를 바탕으로 세부 함수 구축 및 파라미터가 할당되어 유전 알고리즘이 실행된다.

• 가공 데이터(Base_Solution.csv) 속성정의 표

Index	Attributes name	Description	Data Type	Unit
1	Date	생산일자	object	497
2	Produce	생산	object	497
3	Release	출고	object	497



• 주요 변수 기술 통계 (Base_Solution.csv 가공 데이터 활용)

구분	개수(count)	평균(mean)	표준편차(std)	최소값(min)	중간값(50%)	최대값(max)
Date	497	-	-	-	-	-
Produce	497	5005.95	13834.86	12300	41500	71400
Release	497	4924.64	6941.61	1000	10000	37000

• 독립변수/ 종속변수 정의

- 독립변수(Independent Variable)란, 다른 변수에 영향을 받지 않는 변수로 입력값이나 원인을 나타낸다. 독립변수는 입력자에 의해 조정이 가능하며 원인변수(Explanatory Variable), 예측변수(Predictor Variable)라고도 부른다.
- 종속변수(Dependent Variable)란, 독립변수의 변화에 따라 어떻게 변화하는지를 알고 싶어 하는 변수로 결과물이나 효과를 나타낸다. 이로 인해 종속변수를 반응변수(Response Variable), 결과변수(Outcome Variable)라고도 부른다.
- 따라서 독립변수와 종속변수는 원인과 결과의 관계를 맺으며, 해당 데이터의 독립변수와 종속변수는 다음과 같이 구분된다.

공정 변수 조건		내용
독립변수	시간 관련	생산일자, 시작, 종료, 1~10차 출고일
	할당 관련	작업지시번호, 제품코드
	수량 관련	1~10차 출고량
종속변수	수량 관련	생산량

3) 데이터 품질 전처리

• 데이터 품질 전처리 목적

- 실제 공정에서 발생하는 데이터들은 의미가 없거나 누락 및 오타가 발생하여 데이터 품질이 떨어진다.
- 아무리 좋은 분석 방법론을 가지고 있더라도 품질이 낮은 데이터를 활용할 경우 좋은 결과를 보장할 수 없으므로 데이터 품질 전처리는 데이터 분석 과정에서 가장 중요한 단계이다.
- 따라서 데이터 품질 전처리를 위해 아래 6가지 품질 지수를 파악하고, 데이터 전처리를 통해 품질 지수를 향상시킨다.



• 데이터 품질 지수

완전성 (Completeness)	필수항목에 누락이 없어야 한다.
유일성 (Uniqueness)	데이터 항목은 유일해야 하며 중복되어서는 안된다.
유효성 (Validity)	데이터 항목은 정해진 유효범위 및 도메인을 충족해야 한다.
일관성 (Consistency)	데이터가 지켜야 할 구조, 값, 표현되는 형태가 일관되게 정의되고, 서로 일치해야 한다.
정확성 (Accuracy)	실제 존재하는 객체의 표현 값이 정확하게 반영이 되어야 한다.
무결성 (Integrity)	데이터베이스 자료의 오류 없이 변화에 영향을 받지 않고 데이터의 유일성, 유효성, 일관성이 보호되어야 한다.

[표 1] 6가지 데이터 품질지수

• 데이터 품질지수 세부 설명

- 완전성 품질지수

$$\text{완전성 품질지수} = \left(1 - \frac{\text{결측데이터의 개수}}{\text{전체 데이터의 개수}} \right) \times 100$$

- ① 결측 데이터의 값(Null)이 30% 이상인 데이터들은 데이터의 완전성을 떨어뜨리기 때문에 각 열(column)에 대한 결측 데이터값의 비율을 확인하여 삭제한다.
- ② 총 결측 데이터의 개수를 확인하기 위해 'isnull()' 함수를 사용하여 각 열(column)에서의 결측 데이터 개수를 구한 뒤 'sum()' 함수를 이용하여 합산한다.
- ③ 앞서 구한 결측 데이터의 개수를 이용하여 위 식에 따라 완전성 품질지수를 구한다.

- 유일성 품질지수

$$\text{유일성 품질지수} = \left(\frac{\text{유일한 데이터의 개수}}{\text{전체 데이터의 개수}} \right) \times 100$$

- ④ 해당 데이터에서 유일한 값을 갖는 열(column)을 찾는다. (EX) 시계열 데이터 : 시간
- ⑤ 유일한 값을 갖는 열(column)에 대해 'groupby()' 함수를 이용하여, 각 열(column)에서의 유일한 데이터 값을 확인하고, 'size()' 함수를 이용하여 그 개수를 구한다.
- ⑥ 앞서 구한 유일한 데이터의 개수로부터 위 식에 따라 유일성 품질지수를 구한다.

- 유효성 품질지수

$$\text{유효성 품질 지수} = \left(\frac{\text{유효성을 만족한 데이터의 개수}}{\text{전체 데이터의 개수}} \right) \times 100$$



- ⑦ 'max()', 'min()' 함수를 이용하여 데이터가 유효범위를 벗어나지 않는지를 확인한다.
- ⑧ Pandas의 '*.info()' 함수로 데이터의 타입이 정해진 형식에 맞는지 확인한다.
- ⑨ 그 외에 수집된 날짜 안에 들어가 있는지 등을 검증하여 유효성을 만족한 데이터의 개수를 구해 위 식에 따라 유효성 품질지수를 구한다.

- 일관성 품질지수

$$\text{일관성 품질지수} = \left(\frac{\text{일관성을 만족한 데이터의 개수}}{\text{전체 데이터의 개수}} \right) \times 100$$

- ⑩ 데이터 테이블 간 종속관계를 확인한다.
- ⑪ Pandas의 '*.info()' 함수로 데이터의 타입이 정해진 형식에 맞는지 확인한다.
- ⑫ 그 외에 수집된 날짜 안에 들어가 있는지 등을 검증하여 일관성을 만족한 데이터의 개수를 구해 위 식에 따라 일관성 품질지수를 구한다.

- 정확성 품질지수

$$\text{정확성 품질지수} = 1 - \left(\frac{\text{정확성에 위배되는 데이터의 개수}}{\text{전체 데이터의 개수}} \right) \times 100$$

- ⑬ 데이터의 '평균값'이 수집된 데이터에 한하여, 다음과 같이 정확성 품질지수를 확인할 수 있다.
- ⑭ 누락된 데이터가 있는지 확인하기 위해, '*.mean()' 함수를 이용하여 데이터들의 평균을 구한다.
- ⑮ '*.mean()' 함수로 구한 데이터의 평균에서 데이터의 '평균값'과의 차이를 구한다.
- ⑯ 이 차이가 0이 아니면 데이터를 위배하는 데이터가 존재하게 된다.

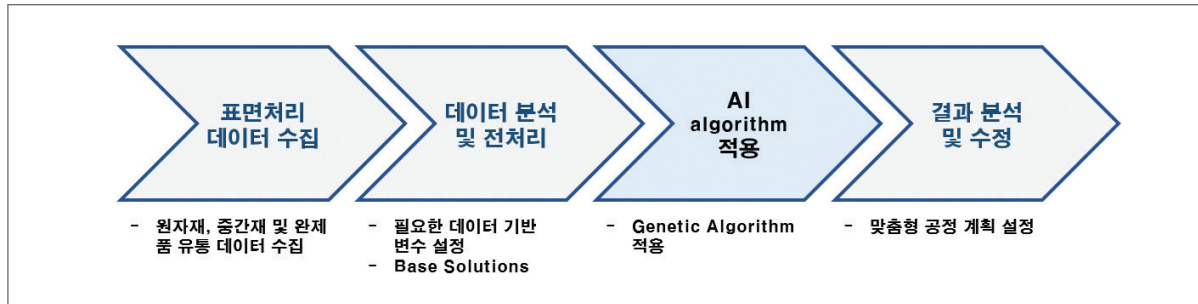
- 무결성 품질지수

$$\text{무결성 품질지수} = \left\{ 1 - \left(\frac{\text{유일성, 유효성, 일관성 중 100%가 아닌 지수}}{3} \right) \right\} \times 100$$

- ⑰ 무결성 품질지수는 데이터가 '유일성', '유효성', '일관성' 지수를 만족하는지 확인하는 지표이다. 일반적으로 세 가지 지수 중 100% 품질을 담당하는 지수의 비율을 무결성 품질지수로 정의한다.

2.2 분석 모델 소개

1) 데이터 흐름 및 인공지능 모델 적용 흐름도



[그림 7] 공정에서의 데이터 흐름 및 AI 모델 적용 단계 도식화

2) AI 분석모델

• 해당 AI 방법론(알고리즘) 선정 이유

- 해당 문제는 출고일에 맞춘 주력 제품에 대한 최적의 생산량을 예측하는 1차 목적을 가진다. 또한, 예측된 생산량을 고려하여 원자재 부족으로 인한 납기일 지연 문제를 해결하고자 최적의 생산 스케줄링을 계획하는 2차 목적을 가진다. 복잡한 유통 구조에 유연한 수주, 발주 공급이 원활하도록 정보를 제공하기 위한 조합 최적화 (combinatorial optimization) 문제이다.
- 생산된 물건 단위는 정수 조건(integer constraints: 모델의 해가 정수가 되는 제약 조건)이 필요하므로 이를 고려한 조합 최적화 문제는 정수계획법(integer programming)이며 다항 시간 내에 문제를 풀 수 없는 NP-hard 문제이다.
- 정수계획법 문제를 해결하기 위한 대표적인 방법론에는 수치 해석적 방법론 (mathematical programming)과 휴리스틱한 방법론(heuristic programming)이 있다. 수치해석적 방법론은 해를 구할 수 있다면 전사적 최적해(global optimal solution)가 보장된다는 장점이 있다. 하지만 문제 구조에 따라 혹은 크기에 따라 어떠한 해도 도출하지 못할 가능성이 있다. 휴리스틱한 방법론은 전사적 최적 해를 보장하지는 않지만, 문제 크기나 구조에 상관없이 최적 해에 근접한 해를 효과적으로 도출할 수 있다. 또한, 목적함수의 수치해석을 이용하지 않기 때문에, 수치 해석적 최적 해를 찾기 힘든 목적함수나 제약조건을 가지는 문제의 솔루션 도출에 효과적이다.
- 따라서 본 가이드북에서는 생산공정 스케줄링뿐만 아니라 다양한 정수 조합 최적화 문제에 적용할 수 있는 확장성을 위해 메타 휴리스틱 알고리즘 중 하나인 유전 알고리즘을 선정하였다.

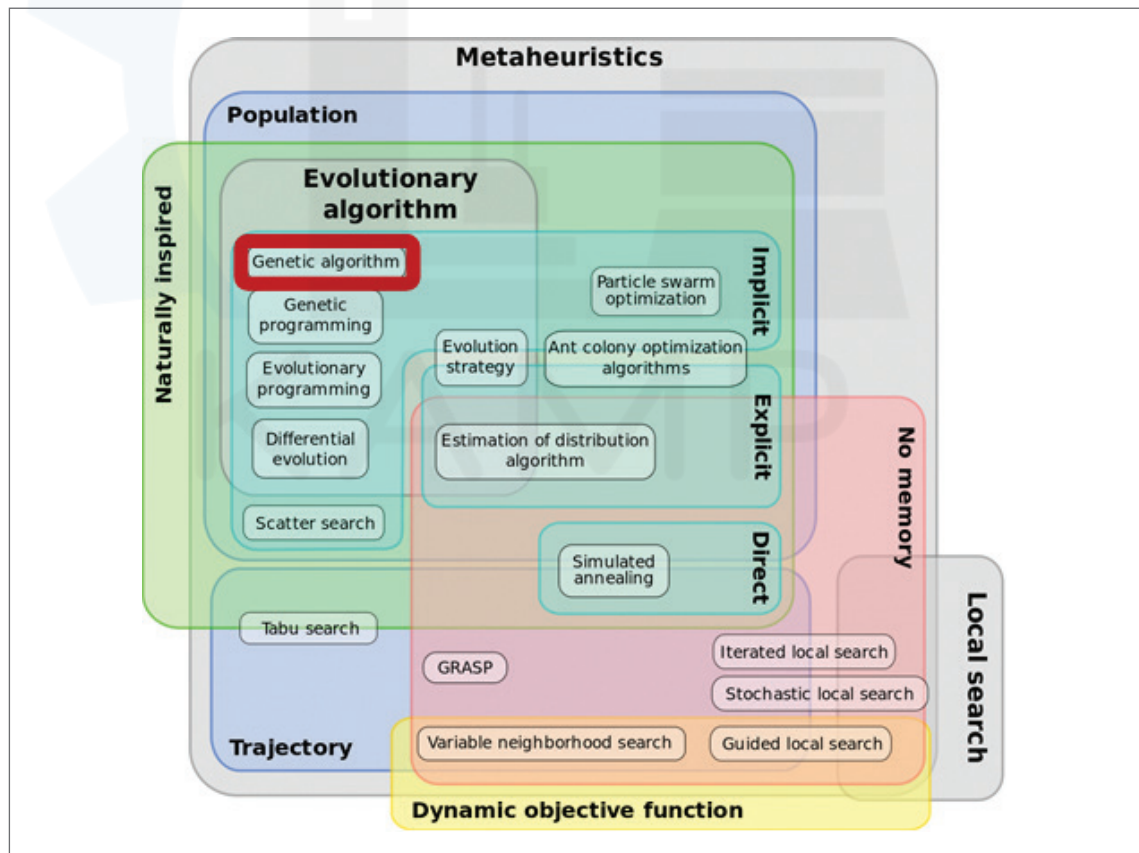
• 적용하고자 하는 AI 분석 방법론(알고리즘)의 구체적 소개

- 유전 알고리즘(Genetic Algorithm) 개념

적자생존, 돌연변이 등 유전 원리에 따라 최적화 형태로 진화 과정에 기초한 계산 모델이다. 주어진 문제에 대한 해답(solution)을 무작위로 생성한 뒤 해당 해답 집단을 진화시켜 더 좋은 해답을 찾는다. 유전 알고리즘은 집단 기반의 최적화 기법인 진화 알고리즘의 한 종류로 집적 회로 설계, 리보핵산(RNA: Ribonucleic acid) 구조 예측, 인공 신경망 학습 등의 다양한 분야의 최적화 및 탐색 문제에 적용되고 있다.

- 대표적인 메타 휴리스틱(Meta Heuristic) 방법론 및 비교

기존의 최적화 문제를 해결하기 위한 알고리즘으로 [그림 8]과 같이 타부 서치(Tabu search), 시뮬레이티드 어닐링(Simulated Annealing) 등이 하나의 해를 사용하여 이웃해를 탐색하는 방식들이 존재한다. 이들은 지역 최적해에 수렴할 확률이 높지만 그에 비해 유전자 알고리즘은 복수 개의 잠재적인 해의 집단(population)을 이용하여 탐색을 하고 우수한 해들을 다음 세대(generation)로 전달하기 때문에 전역해에 수렴할 확률이 높은 특징을 가진다. 본 가이드북에서는 [표 2]에 따라 계산의 간편성과 전역적 근사해를 찾기 위해 유전 알고리즘을 채택한다.



[그림 8] 다양한 메타휴리스틱 방법론

	타부서치(Tabusearch)	담금질 기법 (Simulated annealing)	유전 알고리즘 (Genetic algorithm)
기원	기억 프로세스 (memory process)	금속 담금질	Natural evolution
통제 변수	- 금지 조건 (Tabucriteria) - 기억 사이즈 (Tabulist size)	- 볼츠만 상수 (Boltzmann constant) - 냉각온도 비율 (cooling controlled rate)	- 한 세대를 구성할 염색체 수 - 총 세대 수 등
특징	- 전역적 근사해를 찾을 수 있음	- 전역적 근사해를 찾을 수 있음	- 전역적 근사해를 찾을 수 있음 - 계산 간편성
한계점	- 문제에 맞는 금지조건 설정이 어려움	- 문제에 맞는 파라미터 설정이 어려움	- 목적함수 복잡도에 따라 계산 시간이 다소 소요될 수 있음

[표 2] 3가지 메타 휴리스틱 비교

- 유전 알고리즘 구성 요소

구성요소	핵심 기능	설명
자손 (offspring)	염색체	- 특정 시간(t)에 존재하던 염색체 - 이전 세대와 비슷한 유전 정보
염색체 (chromosome)	집합, 해	- 유전 물질 집합, 해(solution) 표현
돌연변이 (mutation)	유전 정보 변경	- 교차 연산 후 확률적 변경
유전자 (gene)	유전 정보	- 염색체를 구성하는 하나의 유전 정보
적합도 (fitness)	표현 적합도	- 염색체가 보유한 고유의 값

[표 3] 유전 알고리즘 구성 요소

① Initialize

- 유전 알고리즘을 찾고자 하는 해(Solution)를 유전자(Gene) 및 염색체(Chromosome)로 표현하는 단계와 무작위로 염색체를 생성하여 해집합인 유전체(Population)를 구성하는 단계로 이뤄진다.

② Fitness & ③ Evaluation

- 유전체의 성능을 평가하는 척도이다. 좋은 염색체를 찾아내는 과정을 뜻하는 유전 알고리즘에서는 어떤 것이 좋은 염색체인지 판정하는 것이 중요하다.

④ Crossover

- 주어진 염색체 중 뛰어난 염색체를 선정하여 더 나은 염색체를 만드는 과정이다. 뛰어난 염색체를 바탕으로 더 좋은 염색체를 만들면서 유전 알고리즘은 최적해를 가지게 된다. 해당 단계에서는 두 개의 염색체를 선택하는 과정 및 Crossover 방법을 선택해야 한다.

- 두 염색체를 선택했을 경우 Crossover 연산을 수행한다. Crossover은 다양한 연산이 존재한다.

⑤ Mutation

- 주어진 염색체에서 낮은 확률로 변이를 일으키는 새로운 염색체를 만드는 과정이다. Mutation은 유전 알고리즘의 해가 지역 최적해가 아닌, 전역 최적해를 찾을 수 있도록 해주는 장치로 유전자의 위치를 바꾸거나 값을 수정하는 형태로 진행된다.

⑥ Selection

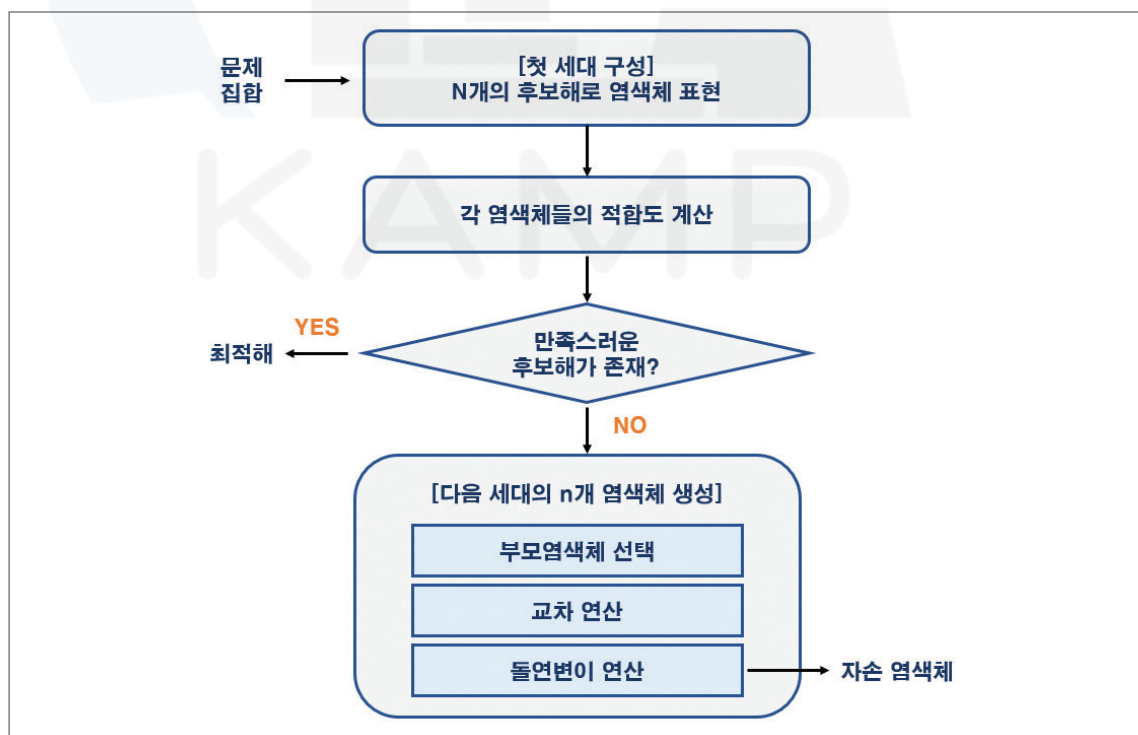
- 해집합인 유전체 속의 염색체에 대하여 높은 적합도를 가지는 염색체가 다음 유전체로 전달될 수 있도록 하여 최적의 해를 찾을 수 있도록 하는 과정이다. 높은 적합도를 가지는 염색체가 다음 세대로 전달되도록 하여 유전 알고리즘이 반복될수록 더 좋은 해를 찾게 한다. 따라서 해당 과정은 유전 알고리즘의 성능에 큰 영향을 미치게 된다. 하지만 어떤 방법을 적용하느냐에 따라 결과가 달라질 수 있으므로 유전 알고리즘의 성능을 끌어올릴 수 있는 중요한 요소로 고려된다.

⑦ Termination

- 유전 알고리즘의 소요시간과 관련된 요인으로서 일반적으로 유전 알고리즘은 최대 세대 수가 생성되거나 모집단에 대해 만족스러운 적합도 수준에 도달하면 종료된다. 또한, 일반적인 유전 알고리즘에는 솔루션 도메인의 유전적 표현, 솔루션 도메인을 평가하기 위한 적합성 함수가 필요하다.

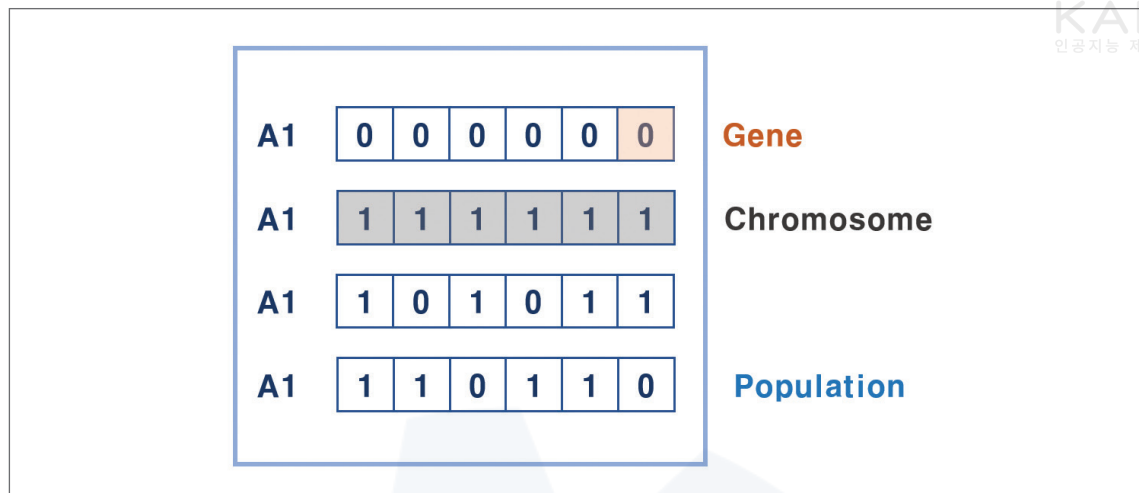
• AI 분석 방법론(알고리즘) 구축 절차

해당 유전 알고리즘은 [그림 9]와 같이 진행된다.



[그림 9] 유전 알고리즘 절차 순서도

▶ Step 1. 초기 임의의 조합해 생성 (Initial Population)



[그림 10] 유전 알고리즘 초기 조합해 생성

- 개인 집합으로 시작되는 Population은 개인 유전자로 알려진 일련의 매개변수(변수)를 특징으로 한다. 유전자는 하나의 끈으로 연결되어 염색체를 형성한다.
- 유전 알고리즘에서 개인 유전자 집합은 문자열을 사용하여 알파벳으로 표현되며 1과 0의 문자열인 이진값으로 사용된다.

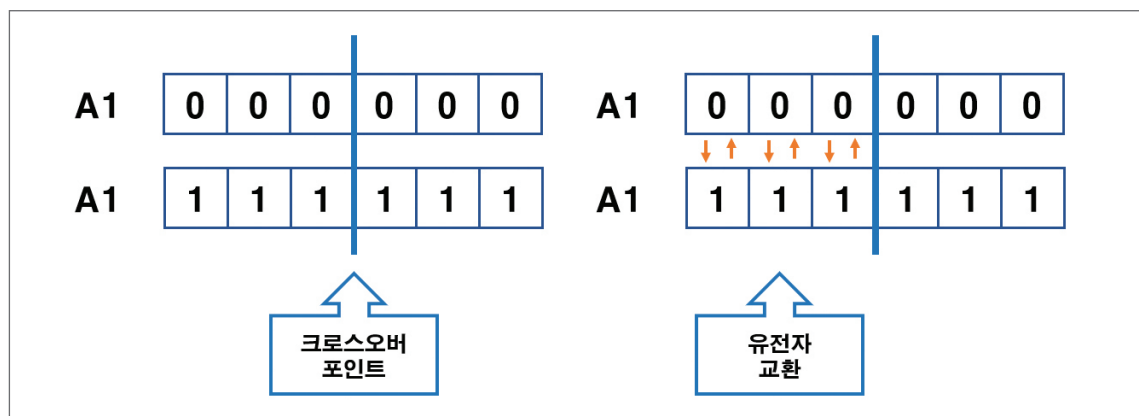
▶ Step 2. 초기 임의의 조합해 적합도 계산 (Fitness)

- 초기 해 집합의 모든 염색체의 적합도를 계산한다. 개체가 번식을 위해 선택될 확률은 해당 개체의 적합성 점수를 기반으로 한다.

▶ Step 3. 적합도 기준 조합해 선택 (Selection)

- 해당 단계에서는 적응도에 따라 생존할 염색체 및 가장 적합한 개체를 선택하여 다음 세대에 유전자를 전달하는 단계이다.

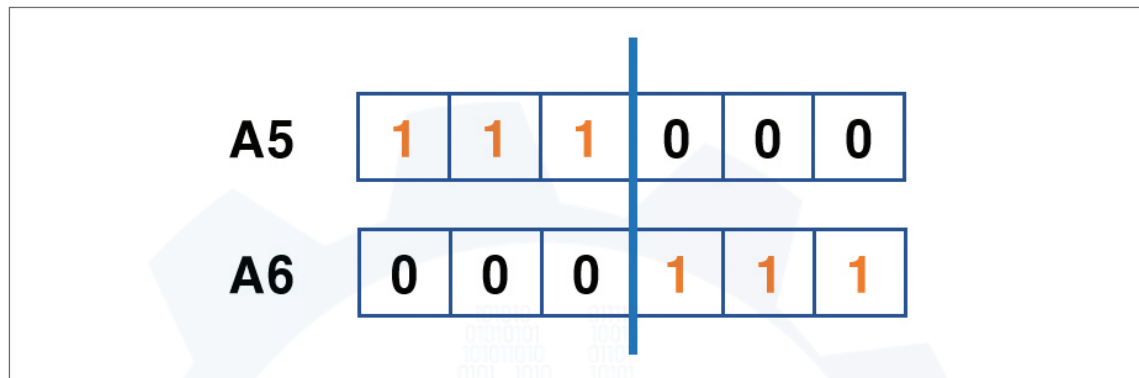
▶ Step 4. 선택된 조합해의 자손해 생성, 재조합 (Crossover)



[그림 11] 재조합 크로스오버 및 유전자 교환 과정

- 유전 알고리즘에서 가장 중요한 단계로서, 다음 세대로 유전자를 전달할 때 해의 다양성 생성을 위해 일정 부분의 유전자를 조합하여 자손을 생성한다.
- 이전 단계에서 선택된 염색체 중에서 임의로 두 염색체를 선택한 후 미리 정해둔 교배율에 맞게 교차시키는 과정이다. 자손은 교차 지점에 도달할 때까지 유전자를 서로 교환하여 생성되며 이후 새로운 자손이 population에 추가된다.

► Step 5. 돌연변이 연산 (Mutation)



[그림 12] 돌연변이 연산 시 새로 추가된 population

- 해당 과정은 다양한 해를 고려하기 위해 재조합 과정 이후 변이율만큼 임의의 유전자를 변경한다.
- 새로운 자손이 형성되면 그들의 유전자 중 일부가 낮은 확률로 돌연변이를 일으킬 수 있다. 개체군 내에서 다양성을 유지하기 위해 발생한다.

► Step 6. 종료 조건 판별

- 이후 미리 정해둔 세대 수에 도달하였거나 특정 종료 조건이 만족 되면 종료한다.

2.3 분석 체험

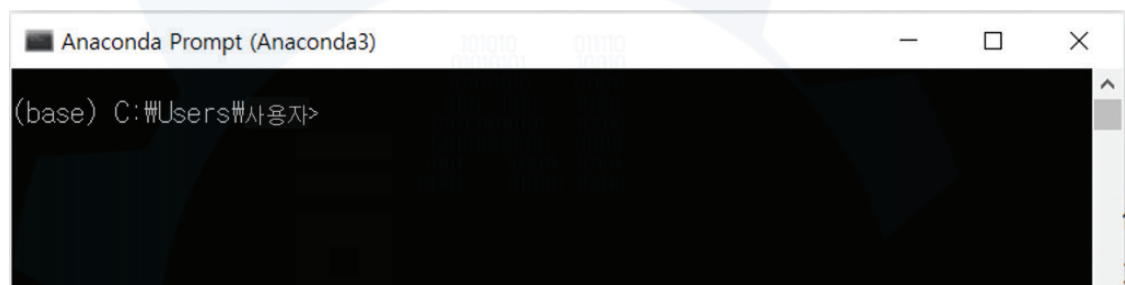
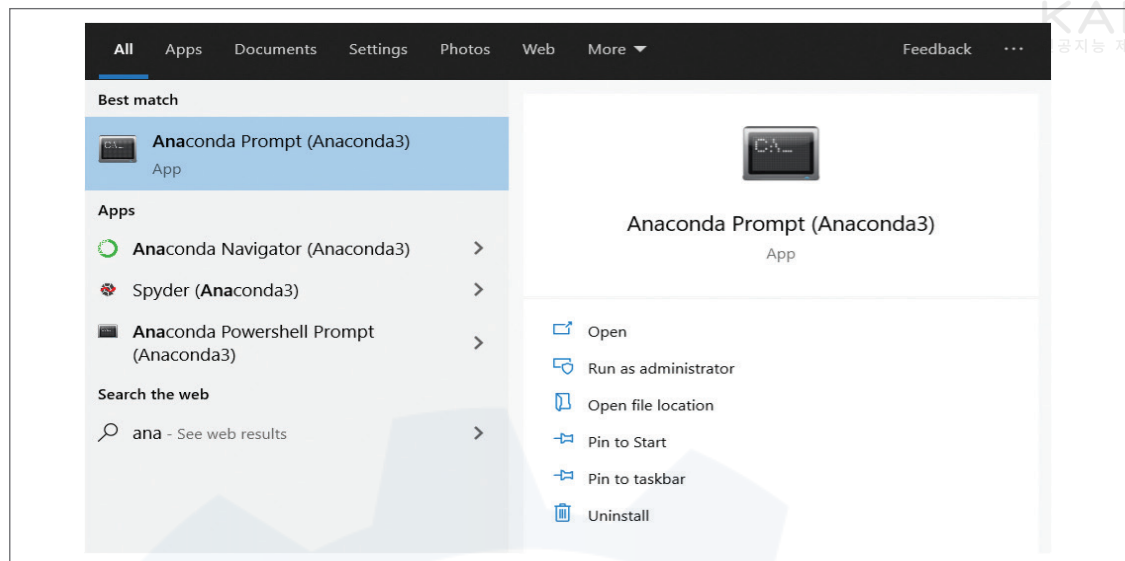
1) 필요 SW, 패키지 설치 방법 및 절차 가이드

► SW 설치는 '부록 3_분석환경 구축을 위한 설치 가이드' 참고

► 가상환경 구축 가이드

- 부록 3. [분석 환경 구축을 위한 설치 가이드]에서 설치한 아나콘다(Anaconda)에서 파이썬의 독립적인 가상의 작업 환경을 구축한다. 파이썬의 모듈은 다양한 버전이 존재하기 때문에 충돌이 일어나기 쉬우며, 이러한 충돌을 방지하기 위해 실습마다 독립적인 가상 환경 구축을 권장한다. 가상 환경을 사용하기 위해서는 환경을 구축하고 실행(활성화)해야하고, 실습이 끝난 이후에는 환경을 비활성화 시켜야 한다. 주피터 노트북에서 가상 환경을 실행하는 구체적인 순서 및 실행 방법은 아래와 같다.

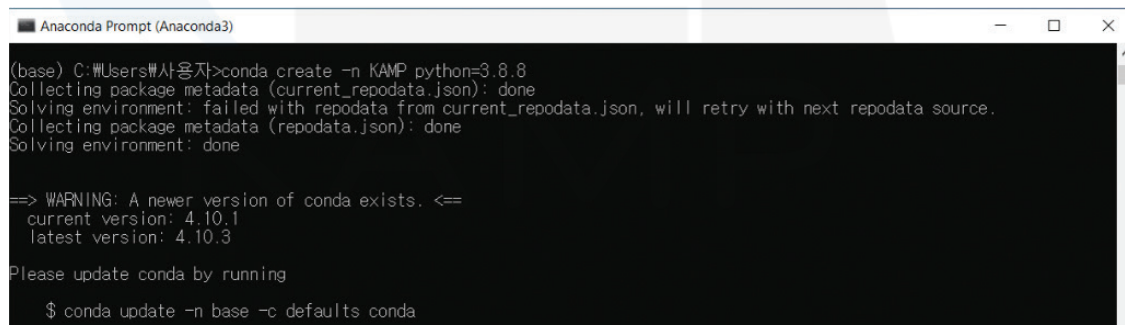
- 아나콘다 프롬프트 실행



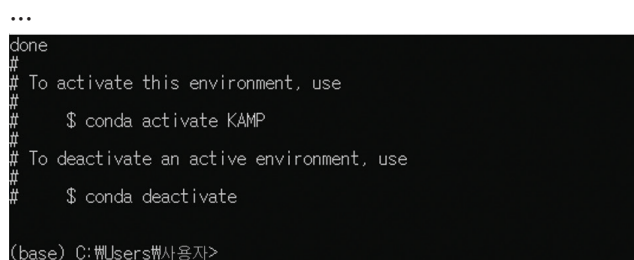
- 가상 환경 생성

```
# conda create -n 가상환경이름 python=버전
```

```
conda create -n KAMP python=3.8.8
```



```
...
Proceed ([y]/n)? y # 해당 문구가 뜨면 'y' 를 입력한다.
```





- 가상 환경 실행

가상 환경이 활성화되면 아래 그림처럼 명령 프롬프트 앞에 (가상 환경 이름)이 표시된다.

```
# conda activate 가상환경이름
```

```
conda activate KAMP
```

```
(base) C:\Users\회사용자>conda activate KAMP
```

```
(KAMP) C:\Users\회사용자>
```

- 분석이 끝난 후에는 가상 환경을 종료한다.

```
conda deactivate
```

```
(KAMP) C:\Users\회사용자>conda deactivate
```

```
(base) C:\Users\회사용자>
```

- 주피터 노트북에서 가상 환경 실행

주피터 노트북에서 가상 환경을 실행하기 위해 가상 환경 비활성화(종료) 상태에서 아래의 코드를 작성한다. 주피터 노트북 실행창에서 빨간 박스처럼 생성한 가상 환경이 있는 것을 확인할 수 있다. 해당 가상 환경을 선택 후 실습을 진행한다.

```
pip install ipykernel
```

```
# python -m ipykernel install --user --name 가상환경이름 --display-name "가상환경이름"
```

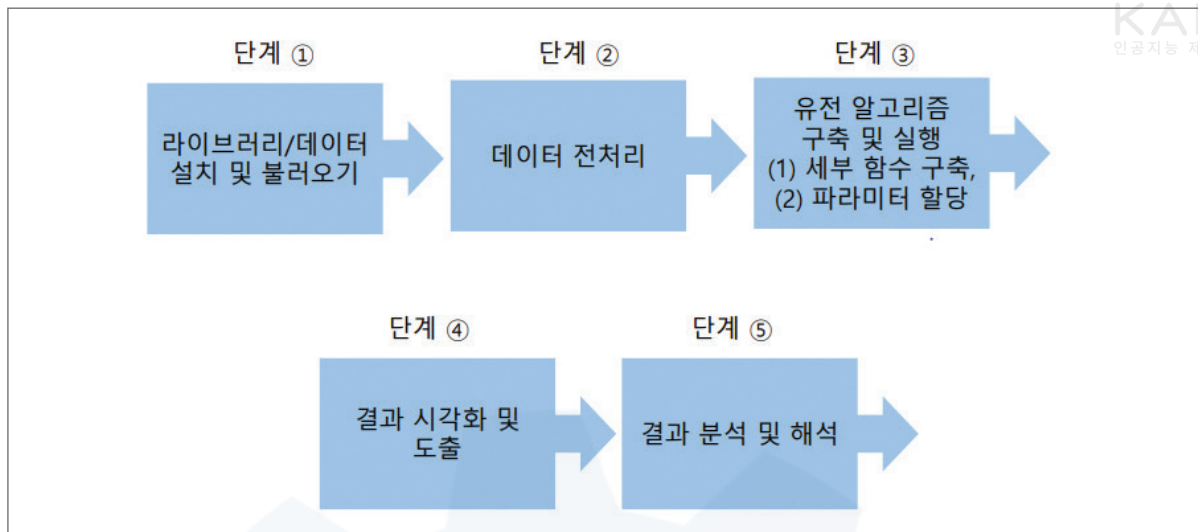
```
python -m ipykernel install --user --name KAMP --display-name "KAMP"
```

```
# jupyter notebook 실행
```

```
jupyter notebook
```



2) 분석 단계별 프로세스 - Flow Chart



[그림 13] 유전 알고리즘 적용을 위한 모델 구현 과정

[단계 ①] 라이브러리/데이터 설치 및 불러오기

①-1. 라이브러리 설치하기

- 유전자 알고리즘을 구축하는 것 외에도 기계 학습 알고리즘을 구축하고 학습할 수 있는 오픈 소스 Python 라이브러리인 PyGAD을 설치한다. PyGAD는 다양한 유형의 교차, 돌연변이 및 상위 선택연산자를 지원하며, 피트니스 기능을 사용자 지정하여 유전 알고리즘을 통해 다양한 유형의 문제를 최적화 할 수 있는 장점이 있다.
- pip을 사용하여 다음 명령으로 PyGAD을 설치한다.
- 이번 분석을 위한 필요 패키지 및 모듈은 pandas, numpy, datetime도 아래와 같이 시작 전에 설치한다.

```
!pip install pygad
```

[코드 1] PyGAD 라이브러리 설치하기

```
!pip install pandas
!pip install numpy
!pip install datetime
```

[코드 2] 필요 패키지 및 모듈 설치하기

①-2. 라이브러리 불러오기

- import <패키지 이름>, 혹은 from <패키지 이름> import <하위 모듈 이름>을 사용하여 전체 패키지를 불러오거나 특정 패키지 안의 모듈만 불러오기가 가능하다. import <패키지 이름> as <약자>를 사용하여 추후 패키지를 사용할 때 이름 대신 약자(alias)로 대신하여 사용할 수 있다.



```
import pandas as pd
import numpy as np
import datetime
import matplotlib.pyplot as plt
import pygad
```

[코드 3] 필요 라이브러리 불러오기

①-3. 데이터 불러오기

- '일무사_표면처리.csv' 데이터를 불러온다. 이때, pandas의 'read_csv("파일 경로")' 기능을 사용하여 csv 파일을 불러온다.

- 불러온 데이터를 아래와 같이 확인할 수 있다.

```
df = pd.read_csv("일무사_표면처리.csv")
df
```

	생산일자	작업지시번호	제품코드	시작	종료	생산량 (Kg)	1차 출고 날짜	2차 출고 날짜	3차 출고 날짜	4차 출고 날짜	...
0	2021-01-12	2101130065	M21	2021-01-13 7:10	2021-01-13 10:40	13,550	2021-01-25	2021-01-26	2021-01-19	2021-01-28	...
1	2021-01-12	2101130066	M21	2021-01-13 10:40	2021-01-13 14:05	14,000	2021-01-19	2021-01-25	NaN	NaN	...
2	2021-01-13	2101130067	M21	2021-01-13 14:05	2021-01-13 17:30	14,000	2021-01-28	2021-02-09	2021-02-17	2021-02-03	...
3	2021-01-13	2101130068	M21	2021-01-13 17:30	2021-01-13 20:40	14,000	2021-01-26	2021-01-25	NaN	NaN	...
4	2021-01-13	2101140075	M21	2021-01-13 20:40	2021-01-13 23:50	15,500	2021-02-17	2021-02-25	2021-02-22	2021-03-12	...
...	...	...	...	...	...	...	...	...	...	...	...
172	2022-04-25	2204250797	M21	2022-04-25 13:30	2022-04-25 17:05	14,000	2022-05-13	2022-05-18	2022-05-02	NaN	...
173	2022-04-25	2204250798	M21	2022-04-25 17:05	2022-04-25 20:19	15,850	2022-05-06	2022-05-11	2022-05-16	NaN	...
174	2022-05-03	2205040846	M21	2022-05-04 4:30	2022-05-04 7:39	14,150	2022-05-16	2022-05-19	NaN	NaN	...
175	2022-05-03	2205040847	M21	2022-05-04 7:39	2022-05-04 10:54	14,000	NaN	NaN	NaN	NaN	...
176	2022-05-03	2205040848	M21	2022-05-04 10:54	2022-05-04 13:57	14,500	2022-05-18	2022-05-19	NaN	NaN	...

177 rows × 26 columns

[코드 4] 데이터 불러오기 및 데이터 확인



[단계 ②] 데이터 전처리

②-1. Column 명 변경 및 데이터 형식 변경

- 각 column명을 영어로 변경한 후, 문자 형식으로 되어 있는 각 column을 정수 형식 혹은 날짜 형식으로 표현되어야 하는 column으로 분류한다.
- 변환된 데이터를 아래와 같이 확인한다.

```
# column명 영어로 변경
df = df.rename(columns={
    '생산일자': 'production_date',
    '작업지시번호': 'order',
    '제품코드': 'item',
    '시작': 'start',
    '종료': 'end',
    '생산량(Kg)': 'production',
    '1차 출고날짜': 'release_date_1',
    '2차 출고날짜': 'release_date_2',
    '3차 출고날짜': 'release_date_3',
    '4차 출고날짜': 'release_date_4',
    '5차 출고날짜': 'release_date_5',
    '6차 출고날짜': 'release_date_6',
    '7차 출고날짜': 'release_date_7',
    '8차 출고날짜': 'release_date_8',
    '9차 출고날짜': 'release_date_9',
    '10차 출고날짜': 'release_date_10',
    '1차 출고량': 'release_amount_1',
    '2차 출고량': 'release_amount_2',
    '3차 출고량': 'release_amount_3',
    '4차 출고량': 'release_amount_4',
    '5차 출고량': 'release_amount_5',
    '6차 출고량': 'release_amount_6',
    '7차 출고량': 'release_amount_7',
    '8차 출고량': 'release_amount_8',
    '9차 출고량': 'release_amount_9',
    '10차 출고량': 'release_amount_10',
})
```

[코드 5] Column 명 영어로 변경하기



```
# 문자 형식에서 정수 형식으로 바뀌어야되는 열
int_columns = ['production', 'release_amount_1', 'release_amount_2', 'release_amount_3',
               'release_amount_4', 'release_amount_5', 'release_amount_6', 'release_amount_7',
               'release_amount_8', 'release_amount_9', 'release_amount_10']

# 문자 형식에서 날짜 형식으로 바뀌어야되는 열
date_columns = ['production_date', 'start', 'end', 'release_date_1', 'release_date_2', 'release_date_3',
                'release_date_4', 'release_date_5', 'release_date_6', 'release_date_7',
                'release_date_8', 'release_date_9', 'release_date_10']

df
```

	production_date	order	item	start	end	production	release_date_1	release_date_2	release_date_3	release_date_4	...
0	2021-01-12	2101130065	M21	2021-01-13	2021-01-13	13550	2021-01-25	2021-01-26	2021-01-19	2021-01-28	...
1	2021-01-12	2101130066	M21	2021-01-13	2021-01-13	14000	2021-01-19	2021-01-25	NaN	NaN	...
2	2021-01-13	2101130067	M21	2021-01-13	2021-01-13	14000	2021-01-28	2021-02-09	2021-02-17	2021-02-03	...
3	2021-01-13	2101130068	M21	2021-01-13	2021-01-13	14000	2021-01-26	2021-01-25	NaN	NaN	...
4	2021-01-13	2101140075	M21	2021-01-13	2021-01-13	15500	2021-02-17	2021-02-25	2021-02-22	2021-03-12	...
...	...	...	...	...	...	...	...	...	...	...	...
172	2022-04-25	2204250797	M21	2022-04-25	2022-04-25	14000	2022-05-13	2022-05-18	2022-05-02	NaN	...
173	2022-04-25	2204250798	M21	2022-04-25	2022-04-25	15850	2022-05-06	2022-05-11	2022-05-16	NaN	...
174	2022-05-03	2205040846	M21	2022-05-04	2022-05-04	14150	2022-05-16	2022-05-19	NaN	NaN	...
175	2022-05-03	2205040847	M21	2022-05-04	2022-05-04	14000	NaN	NaN	NaN	NaN	...
176	2022-05-03	2205040848	M21	2022-05-04	2022-05-04	14500	2022-05-18	2022-05-19	NaN	NaN	...

177 rows x 26 columns

[코드 6] 정수/날짜 형식으로 표현해야 하는 column들로 분류 및 column 명 변환된 데이터 확인하기

②-2. 기준 column의 범위 설정 및 새로운 데이터 형태 도출

- Column 중 'production_date'을 기준 column으로 지정하여 오름차순으로 데이터를 정리하기 위해 생산하는 날짜의 최소값과 최대값을 계산한다.

```
# 생산 가능한 기간을 찾기 위한 날짜의 최소값, 최대값 변수 설정
min_date = datetime.date.fromisoformat(df.iloc[0]['production_date'])
max_date = datetime.date.fromisoformat(df.iloc[0]['production_date'])

for idx, row in df.iterrows():
    # str 타입 -> int 타입으로 변경
    for column in int_columns:
        df.at[idx, column] = int(row[column].replace(", ", ""))
    # str 타입 -> datetime 타입으로 변경
    for column in date_columns:
        if row[column] is not np.NaN:
            df.at[idx, column] = datetime.date.fromisoformat(row[column][:10])
            # 날짜의 최소값, 최대값 계산
            if df.at[idx, column] < min_date:
                min_date = df.at[idx, column]
            elif df.at[idx, column] > max_date:
                max_date = df.at[idx, column]
```

[코드 7] 기준 column ('production_date')의 최소값과 최대값 계산하기

- 계산된 생산 날짜의 최소값과 최대값을 이용하여 그 사이의 날짜 리스트를 생성한다.
그 후, 각 날짜에 해당하는 생산량과 출고량을 계산한다. ([코드 8])
- 계산된 해당 날짜마다의 생산량과 출고량을 이용하여 ["date", "produce", "release"] 형태의 새로운 데이터를 생성한다. ([코드 9])
- 새로운 dataframe은 아래의 [코드 9]과 같이 확인할 수 있다.
- 생성된 새로운 dataframe을 통해 생산 날짜별 생산량과 출고량을 시각화하여 데이터 값들을 전반적으로 확인할 수 있다.

```
# 날짜의 최소값, 최대값 사이의 날짜 리스트
date_list = [min_date + datetime.timedelta(days=x) for x in range((max_date-min_date).days+1)]
produce_list = []
release_list = []
for date in date_list:
    release = 0
    # 해당 날짜의 생산량 계산
    if len(df[df['start'] == date]) == 0:
        produce_list.append(0)
    else:
        produce_list.append(df[df['start'] == date]['production'].sum())
    # 해당 출고량 계산
    for i in range(1,11):
        release_date = release_date_ + str(i)
        release_amount = release_amount_ + str(i)
        if len(df[df[release_date] == date]) != 0:
            release += df[df[release_date] == date][release_amount].sum()
    release_list.append(release)
```

[코드 8] 날짜 리스트 생성 및 해당 날짜의 생산량과 출고량 계산하기

```
# 새로운 dataframe 생성(pr_df / produce(생산), release(출고) dataframe)
pr_df = pd.DataFrame(list(zip(date_list, produce_list, release_list)), columns=["date", "produce", "release"])

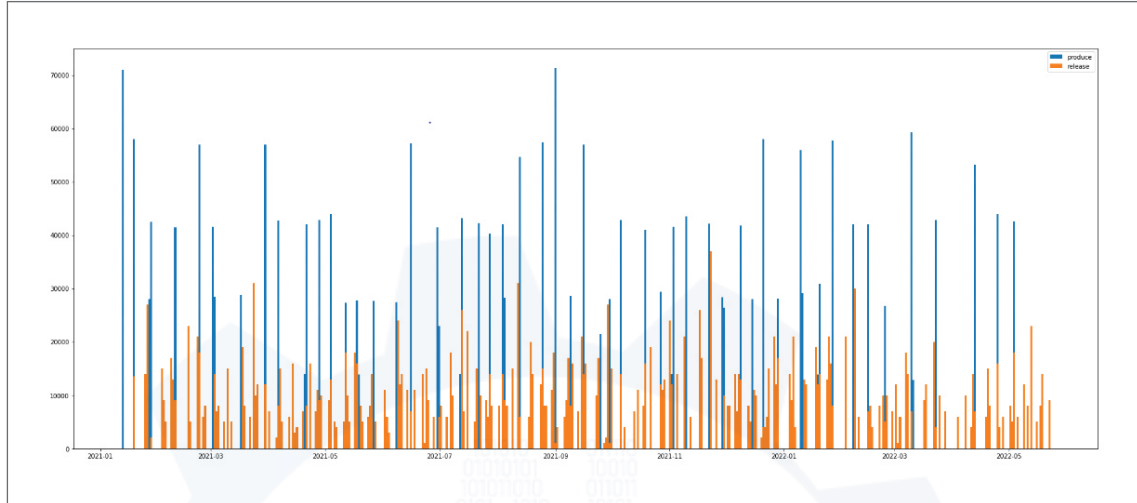
pr_df
```

	date	produce	release
0	2021-01-12	0	0
1	2021-01-13	71050	0
2	2021-01-14	0	0
3	2021-01-15	0	0
4	2021-01-16	0	0
...	...	...	...
492	2022-05-19	0	14000
493	2022-05-20	0	0
494	2022-05-21	0	0
495	2022-05-22	0	0
496	2022-05-23	0	9000

497 rows × 3 columns

[코드 9] 새로운 dataframe 생성 및 확인하기


```
# 새로운 데이터 시각화
plt.figure(figsize=(30,12))
plt.bar(pr_df.date, pr_df.produce, width=1, label='produce')
plt.bar(pr_df.date, pr_df.release, width=1, label='release')
plt.legend()
plt.show()
```



[코드 10] 새로운 dataframe 시각화 및 생산 날짜별 생산량과 출고량

[단계 ③] 유전 알고리즘 구축 및 실행 ((1) 세부 함수 구축, (2) 파라미터 할당)

- 'import pygad'을 통해 불러온 PyGAD로 유전자 알고리즘을 구축한다.
- PyGAD의 매개변수를 먼저 지정한다.
 1. 'inventory_cost': 해당 날짜에 재고해야 하는 양에 패널티를 주는 비율 값, 즉 재고 패널티 비용
 2. 'max': 일일 최대 생산량
 3. 'num_generations': 세대 수
 4. 'num_parents_mating': 세대 당 부모 유전체 채택 수
 5. 'sol_per_pop': 모집단으로써 제안된 솔루션
- 그다음으로, 매우 중요한 단계인 각 솔루션에 대한 적합도 값을 계산하는 데 사용할 'fitness_func' 적합도 함수를 구현한다. 이 때 적합도 함수의 값은 주어진 하루 생산량보다 출고량이 많을 경우, 가능한 출고량만큼 더한 후 부족한 출고량만큼 빼어진다. 예를 들어, 하루 생산량이 100이고 하루 출고량이 150이면 이 때 적합도 함수의 값은 $100 - 50$ 이므로 50이다. 만약 주어진 하루 생산량보다 출고량이 적을 경우, 출고량만큼 더한 다음 남은 재고 양과 재고 패널티 비용을 고려한 값을 제한다. 예를 들어, 하루 생산량이 100이고 하루 출고량이 200이면 이 때 적합도 함수의 값(재고 패널티 비용을 5%라 가정했을 때)은 $100 - 100 * 0.05$ 이므로 95이다.

- 매개변수 및 적합도 함수가 정의되면, pygad.GA 클래스의 인스턴스가 생성된다. 위의 인스턴스에 추가로 사용되는 각 인스턴스에 대한 설명은 다음과 같다.

1. 'num_genes': 솔루션의 유전자 수
2. 'gene_type': 유전자 타입
3. 'init_range_low': 초기 모집단의 유전자 값이 선택된 랜덤 범위의 하한 값
4. 'init_range_high': 초기 모집단의 유전자 값이 선택된 랜덤 범위의 상한 값
5. 'parent_selection_type': 부모 유전체 선택 방식
6. 'mutation_percent_genes': 변이 확률
7. 'mutation_type': 변이 방식

- 인스턴스를 생성한 후 Genetic Algorithm을 실행시킬 수 있다.

```
def genetic_algorithm(inventory_cost, max, num_generations=50, num_parents_mating=5, sol_per_pop=10):
    # fitness 계산
    def fitness_func(solution, solution_idx):
        fitness = 0
        inventory = 0
        for idx in range(len(release_list)):
            # 인벤토리에 생산량 추가
            inventory += solution[idx]
            # 출고량이 있을 때 if문 실행
            if release_list[idx] != 0:
                # 출고량보다 같거나 많은 양을 보유하고 있을 때
                if inventory >= release_list[idx]:
                    # 출고량만큼 fitness를 더해주고 인벤토리에서 출고량을 제함
                    fitness += release_list[idx]
                    inventory -= release_list[idx]
                # 출고량보다 적은 양을 보유하고 있을 때
                else:
                    # 출고 가능한 양만큼 fitness를 더해주고 출고 못 한 양만큼 fitness를 제함
                    fitness += inventory
                    fitness -= release_list[idx] - inventory
                    # 출고량이 인벤토리 재고보다 많기 때문에 인벤토리를 0으로 할당
                    inventory = 0
            # fitness에서 인벤토리 재고량 * 재고 비용을 제함
            fitness -= inventory * inventory_cost
        return fitness

    # Genetic Algorithm 실행
    # 세대 수 = 50, 세대 당 유전체 수 = 10, 세대 당 부모 유전체 채택 수 = 5, 부모 유전체 선택 방식 = "rank", 변이 확률 = 1%, 변이 방식 = "random"
    ga_instance = pygad.GA(num_generations=num_generations,
                           num_parents_mating=num_parents_mating,
                           fitness_func=fitness_func,
                           sol_per_pop=sol_per_pop,
                           num_genes=len(release_list),
                           gene_type=int,
                           init_range_low=0,
                           init_range_high=max,
                           parent_selection_type="rank",
                           mutation_percent_genes=1,
                           mutation_type="random",
                           mutation_by_replacement=True)
    fitness_base = fitness_func(produce_list, 99)
    return ga_instance, fitness_base
```

[코드 11] PyGAD의 Genetic Algorithm 실행



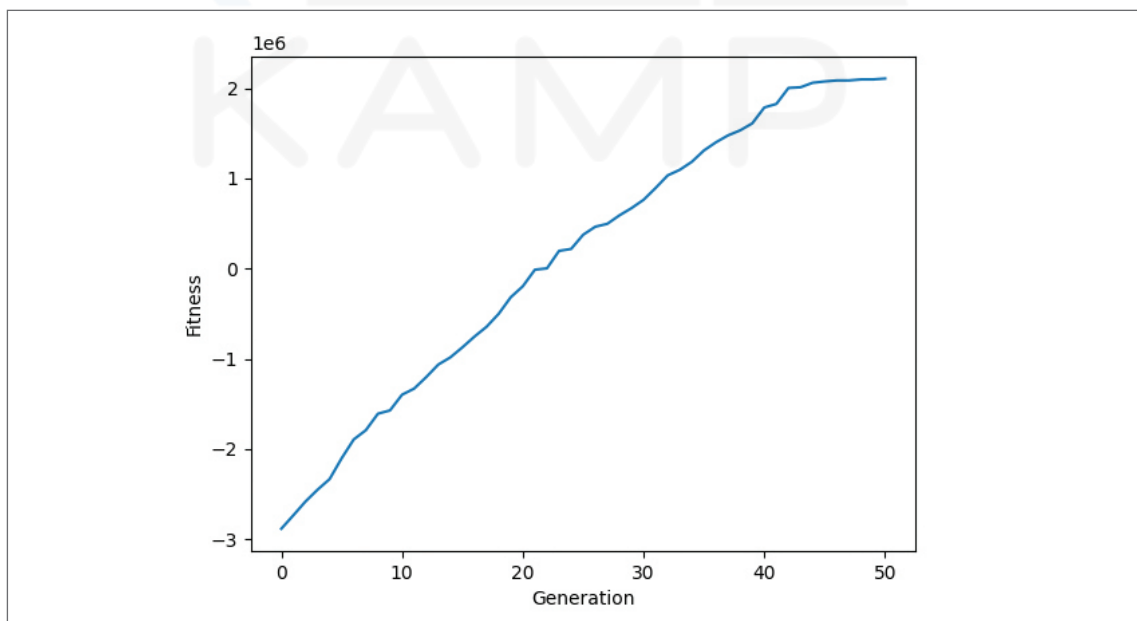
[단계 ④] 결과 시각화 및 도출

- ‘inventory_cost’(재고 패널티 비용)와 ‘max’(일일 최대 생산량)의 파라미터 값에 따라 총 4가지 시나리오로 분류한다.
 1. GA1: ‘inventory_cost’ = 0.01 (1%)와 ‘max’ = 20000 kg 일 때
 2. GA2: ‘inventory_cost’ = 0.05 (5%)와 ‘max’ = 20000 kg 일 때
 3. GA3: ‘inventory_cost’ = 0.01 (1%)와 ‘max’ = 10000 kg 일 때
 4. GA4: ‘inventory_cost’ = 0.05 (5%)와 ‘max’ = 10000 kg 일 때
- 각 시나리오 별 ‘run()’와 ‘plot_fitness()’ 매서드를 호출하여 결과를 시각화한다.
- 제한된 50세대 안에서 각 시나리오별 fitness 값의 변화를 알 수 있다. 시나리오 GA1과 GA2는 세대가 많아짐에 따라 fitness의 값이 증가하고 있다. 시나리오 GA3는 증가했다가 일정해지는 형태를 보여주고 있다. 따라서, GA1, GA2, GA3는 세대수를 늘려 조사하면 최적의 최대 fitness 값을 찾을 것으로 예상된다. 시나리오 GA4는 약 13세대부터는 더 증가하지 않고 일정한 fitness 값을 보여주고 있다.

```
def plot_fitness(solution, file_name):
    plt.plot(solution)
    plt.xlabel("Generation")
    plt.ylabel("Fitness")
    plt.savefig(file_name)
    plt.show()
```

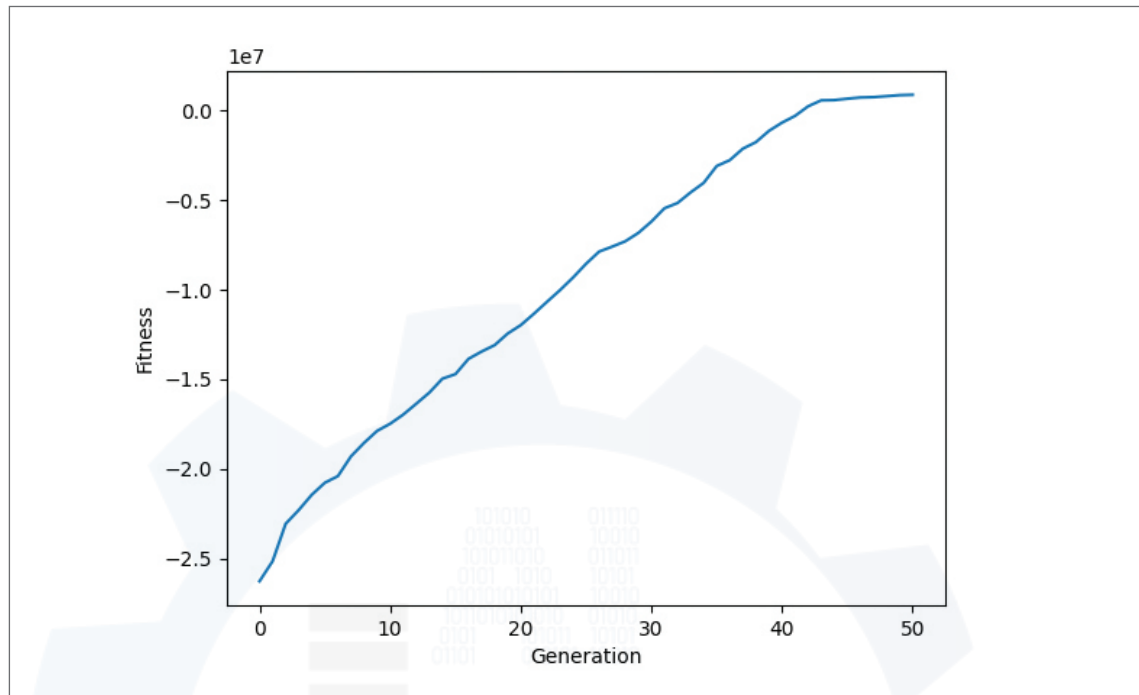
[코드 12] 시나리오 별 결과 시각화를 위한 함수 정의

```
# 각 시나리오 별 결과 시각화
# 재고 비용 = 0.01, 일일 최대 생산량 = 20000kg
ga1, ga1_base = genetic_algorithm(0.01, 20000)
ga1.run()
ga1.plot_fitness(ga1.best_solutions_fitness, "GA1.png")
```



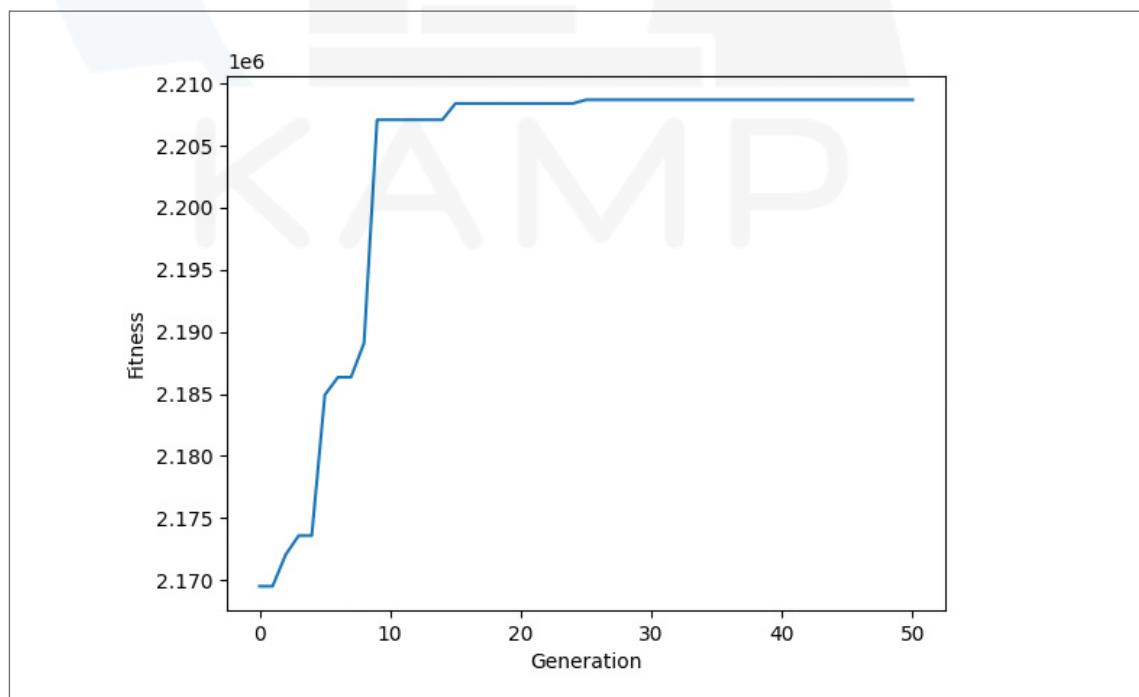
[코드 13] GA1 결과 시각화

```
# 재고 비용 = 0.05, 일일 최대 생산량 = 20000kg
ga2, ga2_base = genetic_algorithm(0.05, 20000)
ga2.run()
ga2.plot_fitness(ga2.best_solutions_fitness, "GA2.png")
```



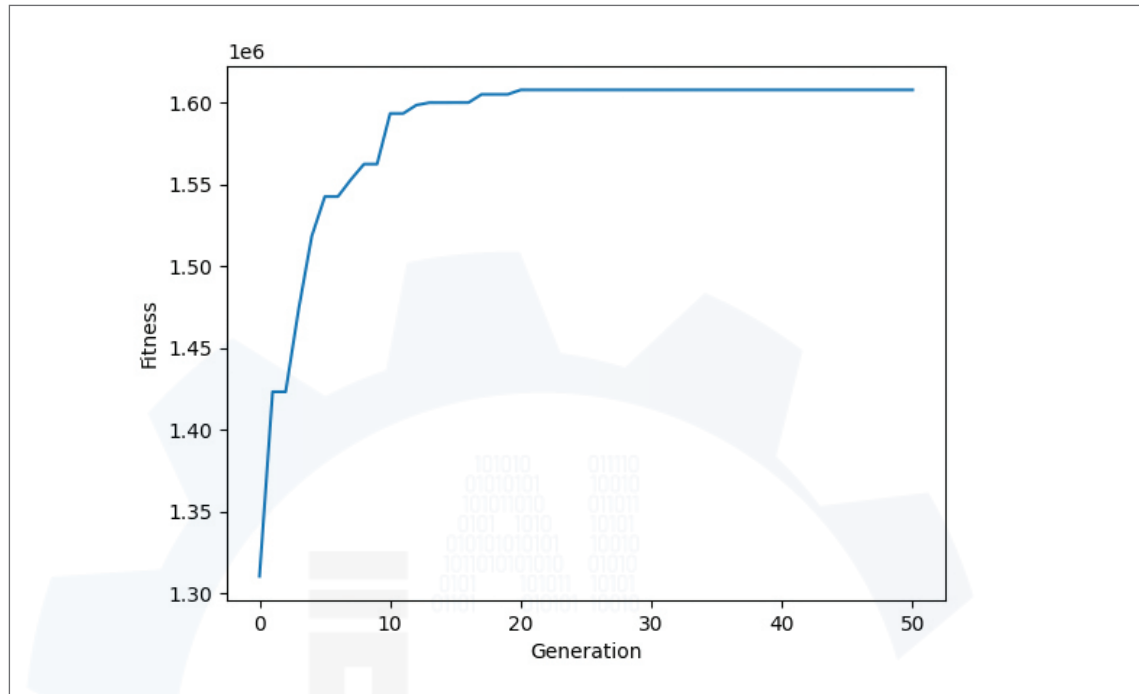
[코드 14] GA2 결과 시각화

```
# 재고 비용 = 0.01, 일일 최대 생산량 = 10000kg
ga3, ga3_base = genetic_algorithm(0.01, 10000)
ga3.run()
ga3.plot_fitness(ga3.best_solutions_fitness, "GA3.png")
```



[코드 15] GA3 결과 시각화

```
# 재고 비용 = 0.05, 일일 최대 생산량 = 10000kg
ga4, ga4_base = genetic_algorithm(0.05, 10000)
ga4.run()
ga4.plot_fitness(ga4.best_solutions_fitness, "GA4.png")
```



[코드 16] GA4 결과 시각화

- 각 시나리오별 'best solutions' 및 결과 값을 도출한다.
- 각 시나리오별 'best solutions'은 각 날짜별 생산해야하는 양을 의미하며, 각 시나리오별 하루 생산량과 재고 패널티 비용에 따른 기존 생산량에 대한 fitness 값 (Fitness value of the base solution)과 best solution에 따른 fitness 값 (Fitness value of the best solution)을 보여주고 있다.
- 따라서, base solution과 best solution의 fitness 값의 차가 클수록 생산량 분배가 잘 되었음을 의미하기 때문에 fitness 값의 차가 큰 시나리오의 파라미터를 선택한다.



```
# 각 시나리오 별 결과 값 도출
# GA1
print("***** GA1 *****")
solution, solution_fitness, solution_idx = ga1.best_solution()
print("Parameters of the best solution : {solution}".format(solution=solution))
print("Fitness value of the best solution = {solution_fitness}".format(solution_fitness=solution_fitness))
print("Fitness value of the base solution = {fitness_base}".format(fitness_base=ga1_base))
print("Index of the best solution : {solution_idx}".format(solution_idx=solution_idx))
```

```
***** GA1 *****
Parameters of the best solution : [ 1445  8446  3760  6951  4528  431 15539    0 12291  5999 18733 2036
 3091    0    0    0    0 11819  1576    0  3756  3630    0    0
 823 15592    0  4236  4597  8194    0    0 11174  1177    0    0
11651 10711 13048    0    0 17964  6189  3744    0  6557  7440  7683
    0    0    0  6247 19013  7495 13952 18628  3418  236 11392  3128
    0    0  9737    0    0   688 12393  2068  2414  172    0 15709
1645    0 16340    0   653  9074  1899 1711    0    0  4190    0
8656    0    0    0    0   5153  5966    0 16332 16014 15631    0
    0  8686    0 13595  8472    0  6906    0  1403  8963  3130 1295
    0    0  3661    0    0 15253  6076 1269    0  7135    0    0
2722  4227  5558 16085    0  7639 10744    0  2497    0  4437 19929
1917    0  9507  6777    0 10801    0  1243    0 19789    0 1590
10474  6242    0    0 10753  2289    0    0  9686    0 14995  6586
15708    0  3676    0  5092 15393 17125  2937  9844    0    0    0
2792  5469    0 15432    0 11820  5880  8034 16166  9640 13801  738
4299  6914 15677  6943    0  6694    0  9350  9415    0    0    0
7226    0    0    0  8030  609    0 15055  1973  7179  3220    0
16304  5494    0 14374  133  6752 14033    0    0 10485    0  6634
9125    0  7623    0    0    0 15107    0    0  5378    0  732
    8 15401 11131  3315  5999  2123 14432  3969  5864 19539    0    0
14055  3937 14350    0 11845 12269  6338  8724    0    0  9825  742
    0    0    0  9132 13986 11003 16478    0  9113    0    0 18955
6291    0  7122  8231    0 12746    0   936    0    0  1414    0
1941  4846  3527 17361 15677  8795    595  2308  6241    0 13305 1961
...
2502    0    0  3482 18586]
Fitness value of the best solution = 2106348.5200000023
Fitness value of the base solution = 1945243.3999999927
Index of the best solution : 7
```

[코드 17] GA1의 base solution, best solution 도출



```
# GA2
print("***** GA2 *****")
solution, solution_fitness, solution_idx = ga2.best_solution()
print("Parameters of the best solution : {solution}".format(solution=solution))
print("Fitness value of the best solution = {solution_fitness}".format(solution_fitness=solution_fitness))
print("Fitness value of the base solution = {fitness_base}".format(fitness_base=ga2_base))
print("Index of the best solution : {solution_idx}".format(solution_idx=solution_idx))
```

```
***** GA2 *****
Parameters of the best solution : [ 0 0 0 10365 14903 1572 16164 5085 3795 5592 16092 1617
0 9220 0 4246 0 14694 0 3335 0 566 0 0
7684 15869 2800 0 0 8153 14878 4808 0 0 0 2565
4282 4486 0 1711 0 17324 2339 4845 0 10076 0 13335
16985 0 0 0 9470 17518 7559 10808 0 0 0 0
4523 7446 0 4194 4290 0 19721 4678 3272 0 2604 17426
3873 0 0 0 0 2925 7332 11692 0 11021 9177 0
0 6783 3433 1482 1252 1848 0 11543 0 8019 0 6502
9464 13949 12836 0 10758 12672 5196 0 83 0 0 2287
6285 0 1896 0 0 0 0 2857 0 0 0 0
11254 11121 0 4224 0 15289 18935 3811 0 8474 0 6959
12880 8854 11842 9597 16170 5647 10922 1807 10459 3921 0 0
0 7762 15805 9089 0 7148 0 0 5585 14450 7300 0
4213 13055 0 4869 7310 1852 6831 0 14971 0 0 12981
8752 1519 0 19992 0 8550 0 8907 0 2208 0 15926
0 0 0 0 0 15942 3793 0 12308 0 988 0
6960 0 17178 18258 0 12710 0 0 2288 6648 0 295
17626 0 2859 0 236 0 0 10479 0 0 0 0
0 13072 2479 0 11529 7507 0 6852 14072 11634 12854 0
10375 0 202 10819 3661 12810 0 16826 1850 3740 8331 0
13941 18541 5954 0 0 0 19409 10999 3408 5483 0 4185
0 508 0 4220 3101 0 0 0 16552 0 1527 8268
15012 5463 0 5666 4266 1311 3903 0 0 11566 0 11113
17009 0 11771 15022 0 11685 0 18315 4095 0 11697 2413
...
0 13767 1022 8442 5659]
Fitness value of the best solution = 876809.400000001
Fitness value of the base solution = -63983.0
Index of the best solution : 6
```

[코드 18] GA2의 base solution, best solution 도출



```
# GA3
print("***** GA3 *****")
solution, solution_fitness, solution_idx = ga3.best_solution()
print("Parameters of the best solution : {solution}".format(solution=solution))
print("Fitness value of the best solution = {solution_fitness}".format(solution_fitness=solution_fitness))
print("Fitness value of the base solution = {fitness_base}".format(fitness_base=ga3_base))
print("Index of the best solution : {solution_idx}".format(solution_idx=solution_idx))
```

```
***** GA3 *****
Parameters of the best solution : [3527 9643 9166 5373 6524 5731 9917 4070 2204 2435 3392 968 220 388
9628 1155 7415 5483 3727 8113 873 6604 291 7938 3606 4295 3588 2705
5094 4597 134 4024 5184 362 7486 9615 2876 1257 8269 2456 4707 7679
3452 1201 6393 187 7428 1963 4832 7632 3076 4622 9354 6623 5867 9141
1519 3919 4508 5727 9644 8142 0 3515 3865 455 3399 3181 5233 9285
5079 3928 7470 8831 5438 8902 1472 4918 1969 7047 2134 6689 2209 6299
6376 913 3554 5843 6894 9087 8168 3254 7252 5338 8030 4972 6222 5581
1490 6117 1287 3935 6262 5041 6709 8400 5366 4027 6380 8094 1385 7746
5880 0 277 7842 8755 0 0 7006 5907 2692 2084 7862 7086 6608
0 2930 6746 9055 7144 7381 4142 4099 5255 4571 1548 1114 9583 7151
6946 5702 3928 1814 3507 2982 7431 7965 4069 9110 8991 4007 2518 0
3831 6621 9816 2908 3791 4115 1976 7538 3338 3821 4392 6347 4393 4268
0 7420 7296 8305 571 2978 1951 1188 6590 2758 1939 1411 7301 3987
0 7397 3820 618 7660 4637 9973 6608 2063 219 7081 8874 7372 424
8862 355 8254 3838 2142 1929 9440 5224 2013 6723 1378 8882 1326 5388
9084 7962 2479 2988 2457 8629 8292 8968 3434 9333 8673 6126 8941 9226
8913 9665 3508 7625 9008 3656 7897 3677 3487 1720 4206 7499 5488 7100
3764 6253 7555 398 7607 2562 4098 9462 3499 7278 6149 7264 9494 9986
2191 9292 3326 7795 3615 6034 4780 1920 4581 9070 9704 3840 5794 788
6147 4650 0 3460 9026 7117 528 7241 4281 1951 7201 2557 1607 6576
6687 257 0 5195 9088 1178 4323 5911 3012 201 7805 5799 484 9676
9117 2862 4037 3019 5651 2521 4078 5507 8494 5246 4570 7358 758 7986
5301 9438 1065 3502 3971 5767 781 9346 6806 3994 9661 9645 2764 6315
2501 4181 1438 4091 7231 5265 7061 7248 3848 5547 6391 8272 6406 9204
...
4899 8882 296 358 7892 5723 0]
Fitness value of the best solution = 2208685.620000002
Fitness value of the base solution = 1945243.3999999927
Index of the best solution : 0
```

[코드 19] GA3의 base solution, best solution 도출



```
# GA4
print("***** GA4 *****")
solution, solution_fitness, solution_idx = ga4.best_solution()
print("Parameters of the best solution : {solution}".format(solution=solution))
print("Fitness value of the best solution = {solution_fitness}".format(solution_fitness=solution_fitness))
print("Fitness value of the base solution = {fitness_base}".format(fitness_base=ga4_base))
print("Index of the best solution : {solution_idx}".format(solution_idx=solution_idx))
```

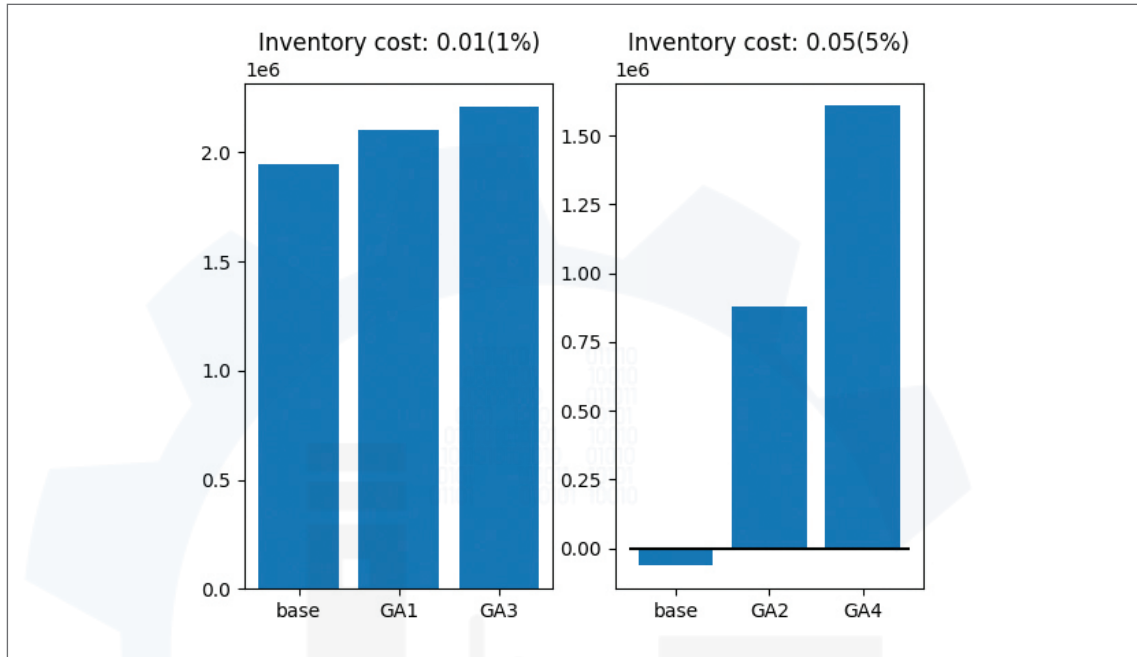
```
***** GA4 *****
Parameters of the best solution : [4108 1175 5553 4857 7945 6181 7133 868 3831 3300 7631 7628 0 6734
2149 5478 6897 5154 6410 5794 5013 3656 0 449 6680 1484 1853 1661
112 4939 6223 2369 0 8619 1703 9752 9893 6504 6284 8740 4674 5918
0 6889 7466 1724 9875 7914 93 0 2034 5884 9400 5712 0 6556
0 1184 7600 90 1221 5495 2555 1775 2044 9961 2605 589 5233 1262
4919 7631 8810 6305 8688 7549 3841 7806 1746 1279 2577 8772 7858 5885
3779 4530 2133 2781 6295 4891 6429 0 5369 4160 73 4591 8716 2346
871 8604 2606 1262 7157 6201 8333 757 6289 4586 2185 4315 3859 1011
1825 0 7301 3827 6850 6711 413 6763 9646 9589 3165 2446 0 9239
2941 585 7651 2680 4228 7905 3590 2193 8311 8824 2723 0 2688 8588
6369 3690 2496 3840 5845 2090 2097 2995 9937 2649 5944 1063 1201 0
3004 2008 9043 787 5586 7105 0 81 2377 9912 4901 9684 1999 9182
6224 2761 3387 7616 6361 4882 812 9886 7708 4301 1130 9026 4606 2694
2365 0 5137 5403 4467 8146 7787 3170 598 7049 7106 9518 2844 5750
348 9097 4246 4795 3471 4246 6984 2220 6319 9452 2205 720 9537 7102
2404 5102 9652 161 4227 4999 945 3497 7607 1255 1595 9549 8681 6472
9936 796 814 5168 1531 9957 3150 3628 0 5974 6438 1273 1978 9563
5537 9530 6886 2988 9995 1529 2380 5437 7243 4546 4764 8620 915 165
1664 1232 9026 9609 7641 0 2999 6521 8456 2147 9741 1089 9029 9494
0 0 4503 6918 150 9929 8347 9777 0 7100 0 104 6922 7662
4199 9078 952 0 2382 9010 8087 9065 2554 0 2037 6006 8629 5818
6686 2183 1571 4177 9374 7735 0 4578 1067 5776 5515 8552 1010 6960
9207 7834 1782 6937 0 3788 171 7992 8582 9071 1283 8325 6136 0
9483 3465 727 8429 8032 1229 4090 5567 2993 5212 0 3879 0 7173
...
2185 5112 4761 4225 8216 7066 6110]
Fitness value of the best solution = 1607675.4499999976
Fitness value of the base solution = -63983.0
Index of the best solution : 0
```

[코드 20] GA4의 base solution, best solution 도출

시나리오	Fitness value	Base solution	Best solution	Difference
GA1		1945243.40	2106348.52	161105.12
GA2		-63983.00	876809.40	940792.40
GA3		1945243.40	2208685.62	263442.22
GA4		-63983.00	1607675.45	1671658.45

[표 4] 각 시나리오별 결과

```
plt.subplot(1, 2, 1)      # nrows=2, ncols=1, index=1
plt.bar(["base", "GA1", "GA3"], [ga1_base, ga1.best_solutions_fitness[-1], ga3.best_solutions_fitness[-1]])
plt.title("Inventory cost: 0.01(1%)")
plt.subplot(1, 2, 2)      # nrows=2, ncols=1, index=2
plt.bar(["base", "GA2", "GA4"], [ga2_base, ga2.best_solutions_fitness[-1], ga4.best_solutions_fitness[-1]])
plt.hlines(0, -0.5, 2.5, colors="black")
plt.title("Inventory cost: 0.05(5%)")
plt.savefig("result.png")
plt.show
```



[코드 21] inventory cost 별 시나리오 결과 비교

[단계 ⑤] 결과 분석 및 해석

- 유전 알고리즘은 임의성이 바탕이며, 이로 인해 결과는 매번 달라진다.
- 위의 시나리오 4개 중 'best solution'과 'base solution'의 값의 차가 가장 큰 파라미터 조합을 선택할 수 있다.
- [단계 ④] 실험을 통해 inventory cost 별 (inventory cost가 1%인 GA1과 GA3, inventory cost가 5%인 GA2와 GA4) 'best solution'과 'base solution'의 값의 차이가 큰 시나리오는 각각 GA3와 GA4이다. 따라서, 이는 inventory cost가 1%일 때는 최대 하루 생산량을 10000kg으로 (GA3), inventory cost가 5%일 때는 최대 하루 생산량을 10000kg으로 (GA4) 선택하는 것이 좋은 조합이다.
- 다만 유전 알고리즘을 이용한 최적해는 정 해(exact solution)를 알 수 없기 때문에 정확성을 평가할 수 없다. 따라서 현재 모델의 최종 해에서 'inventory_cost' (재고 패널티 비용)와 'max' (일일 최대 생산량)를 수정하면서 결과를 개선할 수 있다. 개선 방향은 모델을 사용하는 전문가가 필요로 하는 값들로 지정하거나 변경하며 경험적으로 알아내야 한다.

- 한 세대를 구성하는 구성이 많을수록, 거듭되는 세대가 많을수록 더 좋은 최적 해를 찾는 데에 유리하지만, 최적 해와 계산 비용은 서로 절충 (trade-off) 관계를 맺고 있으므로 이 점을 고려해 파라미터를 결정하는 것도 중요하다.

3. 유사 타 현장의 「표면처리 공급망최적화 AI 데이터셋」 분석 적용

3.1 본 분석이 적용 가능한 제조현장 소개

- 본 가이드북에서는 생산 및 유통 데이터를 바탕으로 AI 기반의 최적의 생산량 예측 모델을 구축했다. 공급망 최적화 기술은 생산량 예측 뿐만 아니라 다양한 목적으로 적용이 가능하다.
- 데이터 분석을 통해 공급망을 구성하는 다양한 변수들 중 각 기업의 설정 기준에 따라 생산량이 아닌 투입량과 같은 독립 변수로 설정 및 적용하는데 사용될 수 있다.
- 또한, 복잡한 유통 구조 및 고객 주문 수요와 같이 급격하게 변화하는 제조업 시장에서 유연한 대응성을 기대할 수 있다. 이를 위해 변동에 예민한 수주, 발주 및 원자재 공급 데이터 수집을 통해, 이를 바탕으로 재고 관리 및 공급망 최적화에 대한 합리적인 생산계획 및 전략을 수립할 수 있다. 이는 고객 수요와 기업의 제조 능력을 기반으로 진행된다.

3.2 본 분석을 원용하여 타 제조현장 적용시, 주요 고려사항

- 본 가이드북에서는 예측하고자 하는 변수를 선정하는데 있어서 여러 가지 가정들을 바탕으로 데이터를 전처리하는 과정이 요구되었다. 각 적용 현장의 목적에 따라 적절한 변수를 설정하기 위해, 이처럼 필요시 양질의 데이터를 새롭게 수집하거나 불필요한 데이터를 제거하는 과정이 요구된다.
- 유통 구조상 공급 데이터가 수집될 때 데이터의 품질을 확인하는 것이 요구된다. 특히 비정상 데이터가 많이 포함된 정보는 분석모델의 예측 정확도에 영향을 주기 때문이다. 데이터 신뢰를 기반으로 변수들 간의 연관성을 명확히 하는 것이 특히 중요하다.
- 수집하는 데이터의 형태에 대한 논의 및 관리가 필요하다. 현재는, 현장에서 수집되는 데이터와 AI 분석을 위한 데이터에는 많은 전처리 과정이 필요하다. 데이터 수집을 하면서 현장 실무자와 AI 분석 전문가가 꾸준히 커뮤니케이션하면서 데이터 수집 방향 및 형태에 대해서 논의하여 수집 환경을 구축하는 것이 중요하다.
- 본 가이드북에서 활용한 특정 변수는 타 제조현장의 상황과 맞지 않을 수 있으므로 현장 실무자 및 AI 분석 전문가의 의견을 반영하여 적용 여부를 결정하는 것이 중요하다.

3 부록 _ 분석환경 구축을 위한 설치 가이드

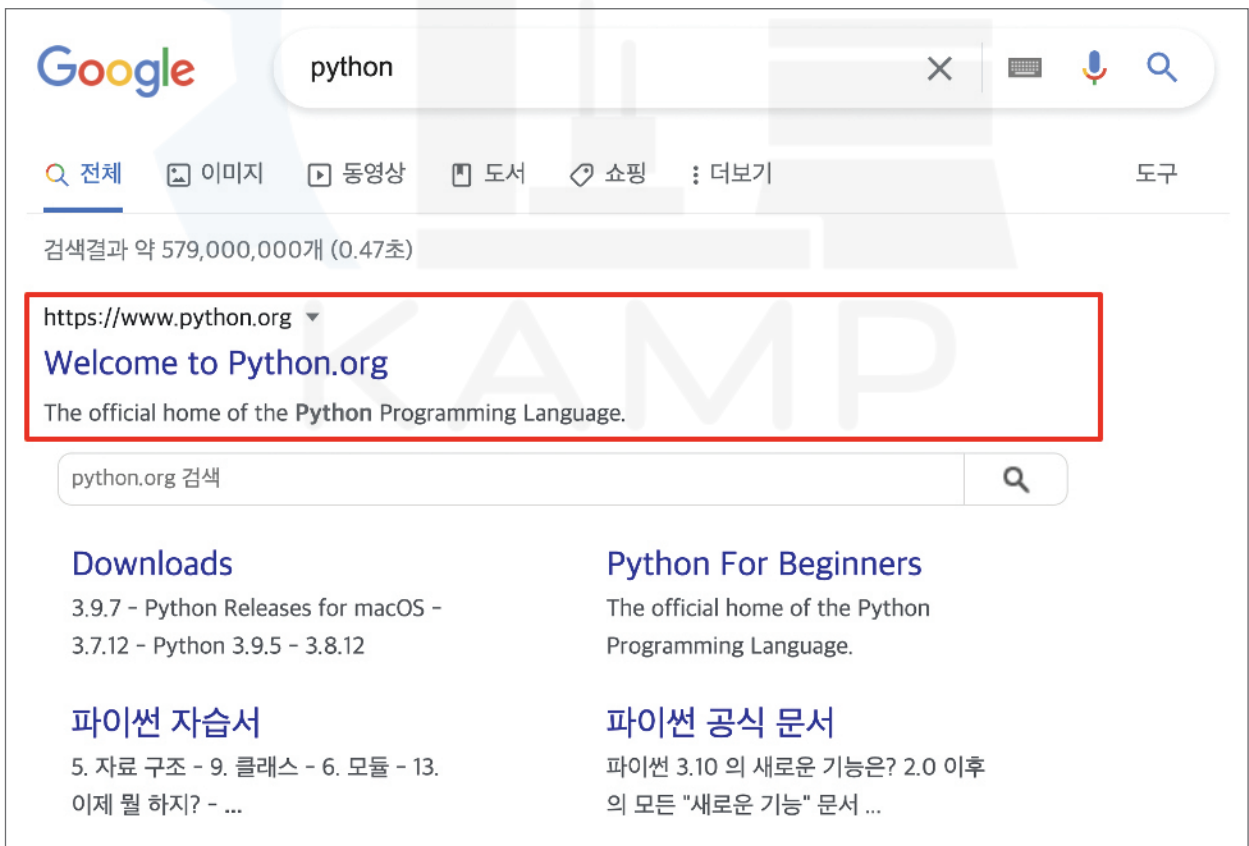
1. 파이썬(Python) 설치

파이썬은 컴퓨터 프로그래밍 언어로서, 데이터 분석 및 AI 분석모델 개발 등에 널리 사용되는 도구이다. 다운로드 및 설치가 간편하고 활용도가 높은 파이썬을 로컬 컴퓨터에 설치하는 방법을 안내한다.

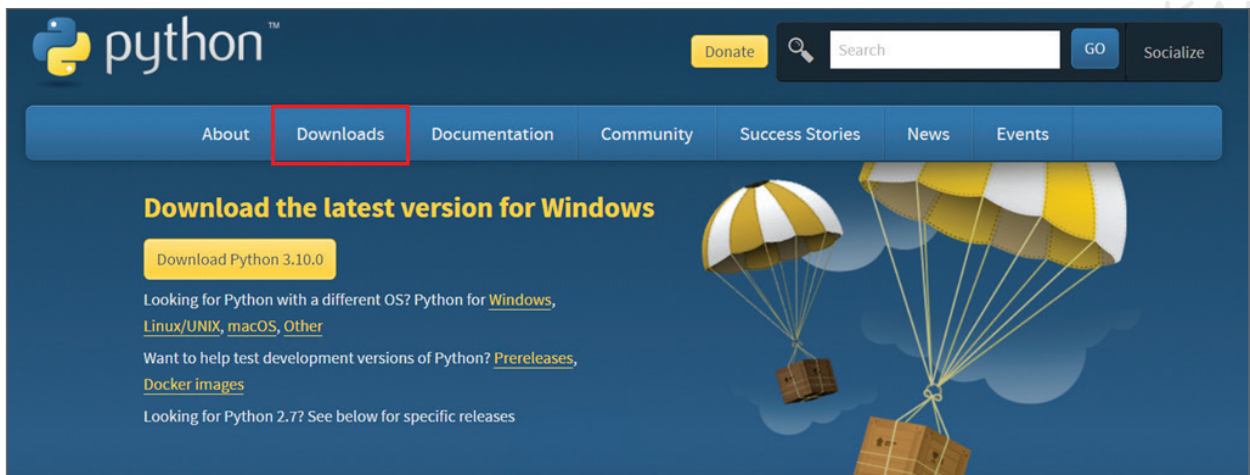
① google.com 등의 검색 엔진에 'python'을 검색한다.



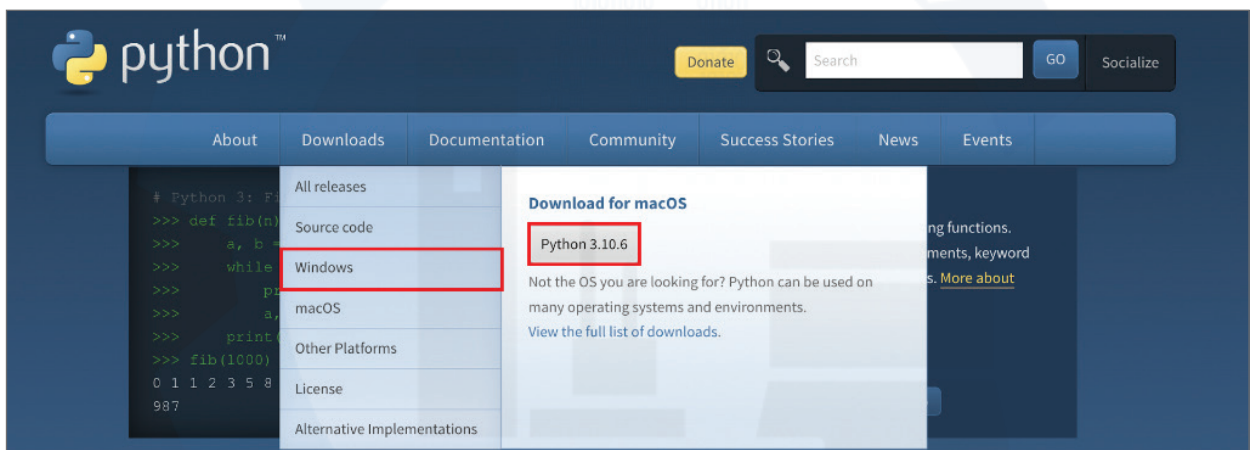
② 제일 처음에 보이는 'Welcome to Python.org'를 클릭한다.



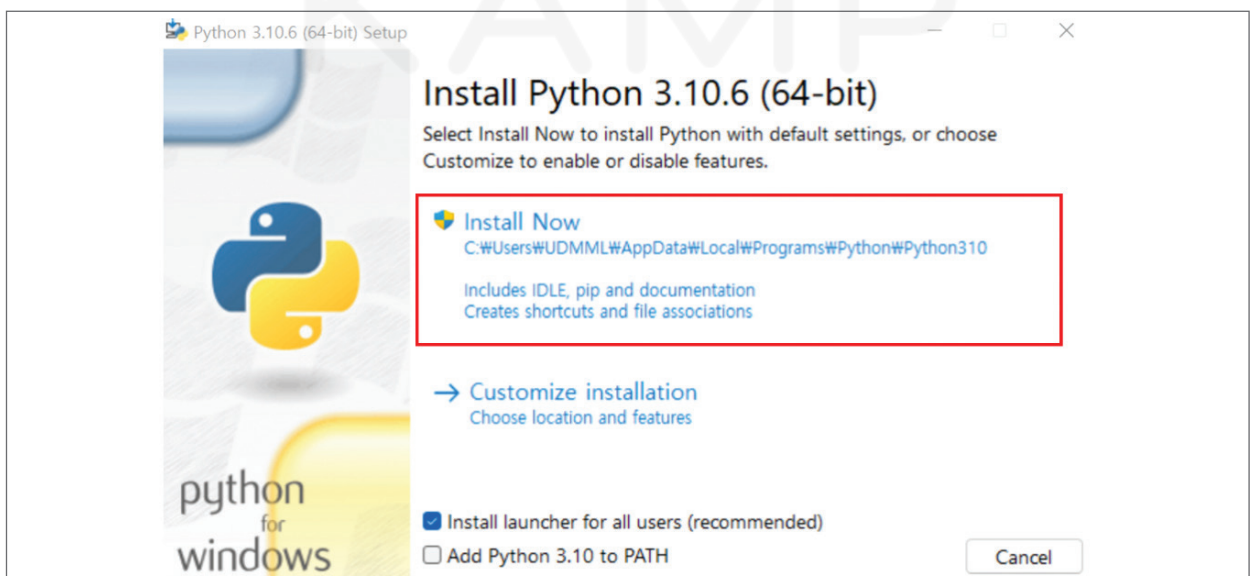
③ 클릭 후 보이는 페이지의 정면에서 상단 좌측 2번째 'Downloads'를 클릭한다.



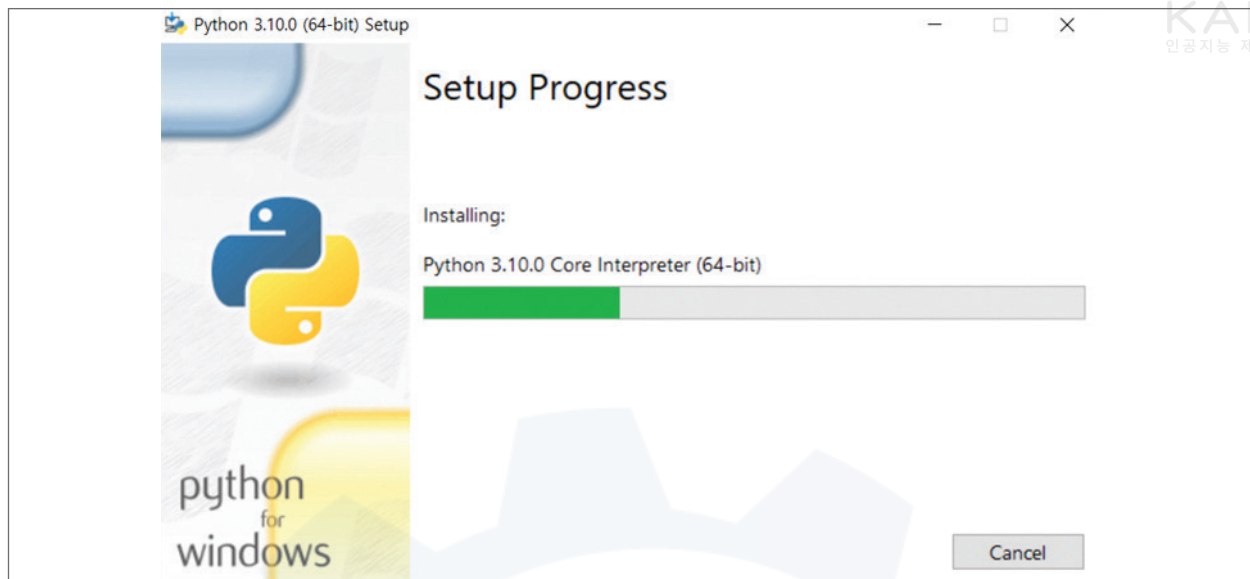
④ 컴퓨터의 운영체제에 따라 상단에서 세번째(macOS의 경우 네번째) 탭을 선택한 후, Python 3.10.0을 다운로드한다. (Python 버전은 업데이트 될 수 있다.)



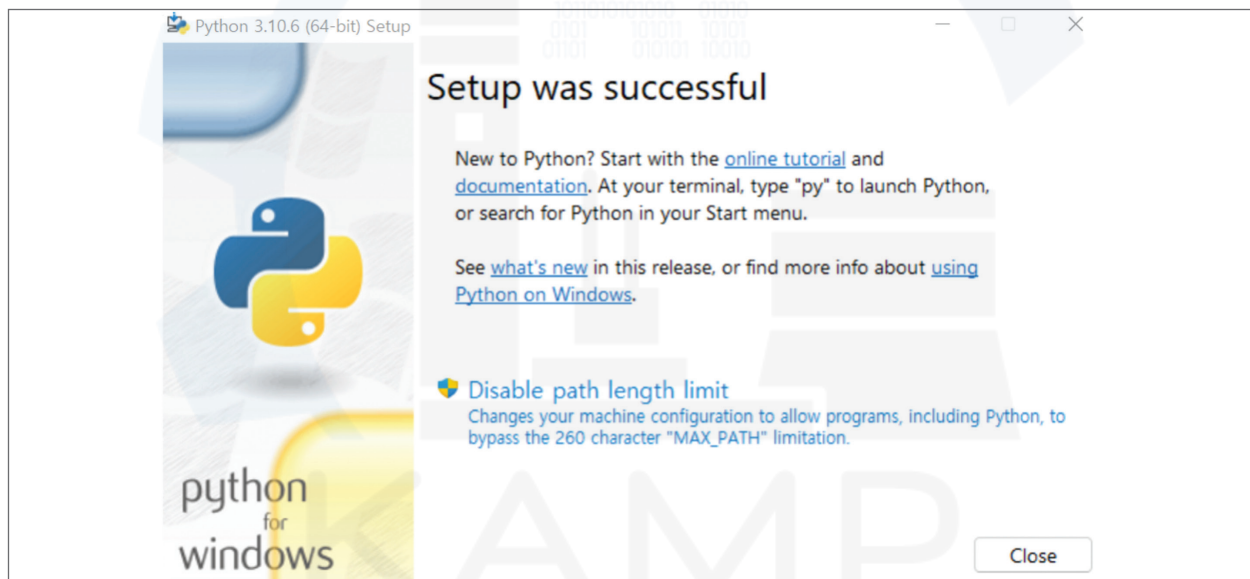
⑤ 아래와 같은 설치창이 뜨면, 'Install Now'를 클릭한다.



⑥ 아래와 같은 설치 진행창이 완료가 될 때까지 유지한다.



⑦ 아래와 같은 창이 나타나면 설치 완료된 것으로, 확인 후 종료한다. [설치완료]

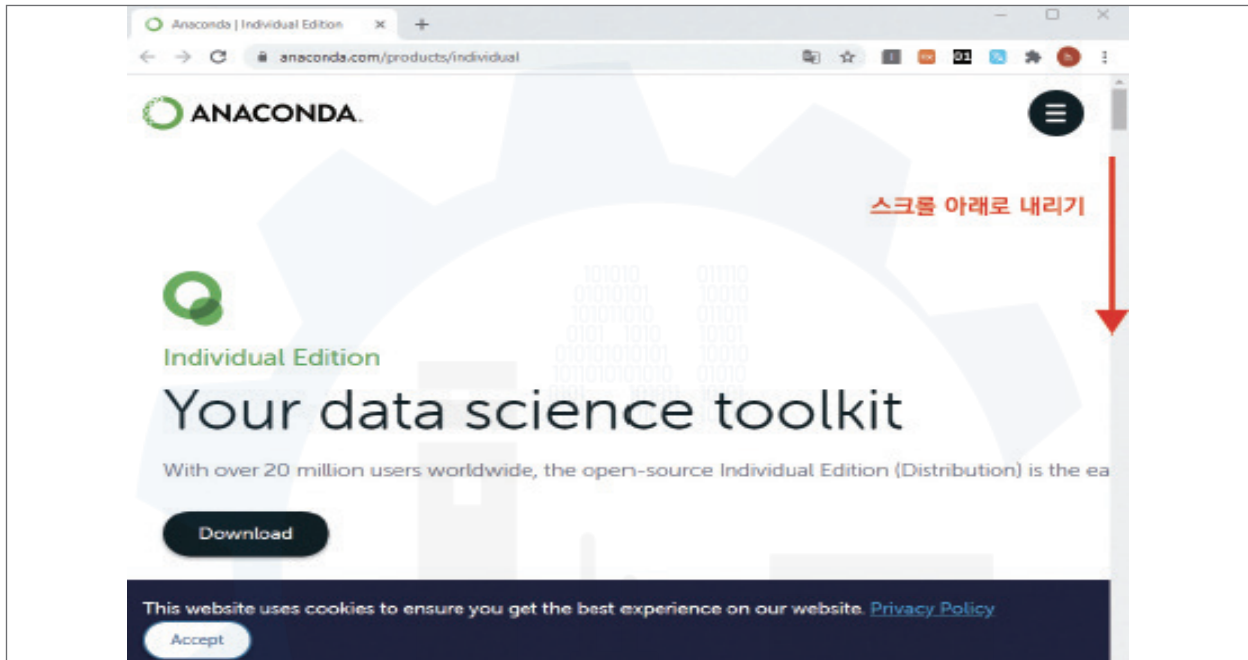




2. 아나콘다(Anaconda) 설치

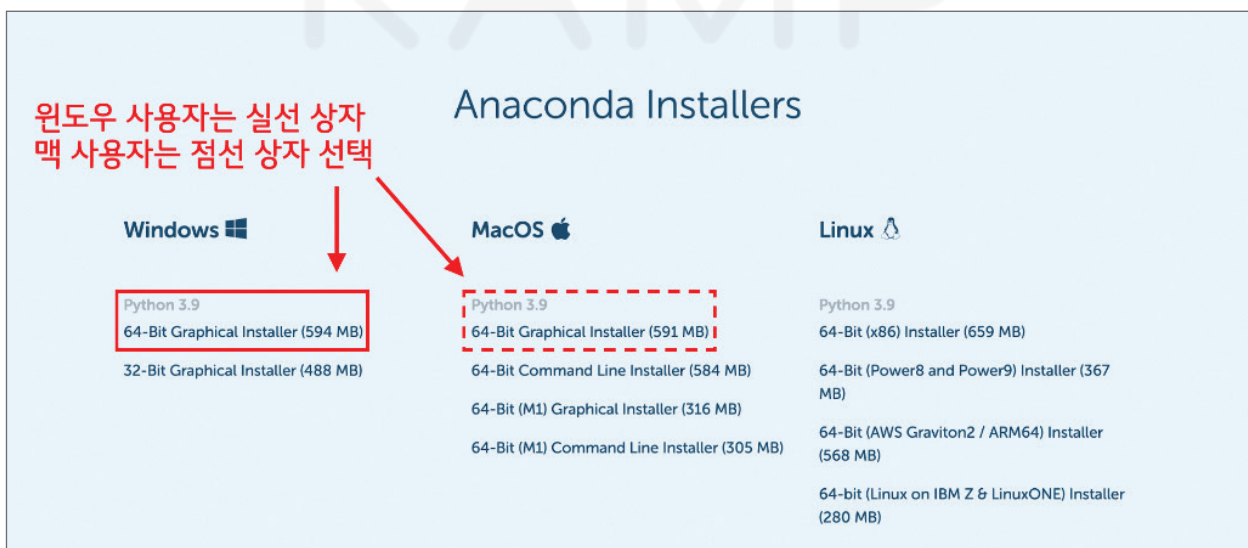
- 아나콘다란 파이썬 등의 데이터 분석 도구를 사용할 때 필요한 고급 기능 및 분석을 보조하는 도구이다. 아나콘다를 설치함으로써 다양한 기능들을 효율적으로 사용할 수 있으며, 결과물을 쉽게 확인할 수 있다. 아나콘다를 설치하고, 데이터를 분석할 수 있는 환경을 구축하는 방법에 대해 안내한다.

① <https://www.anaconda.com/distribution/> 로 접속한다.



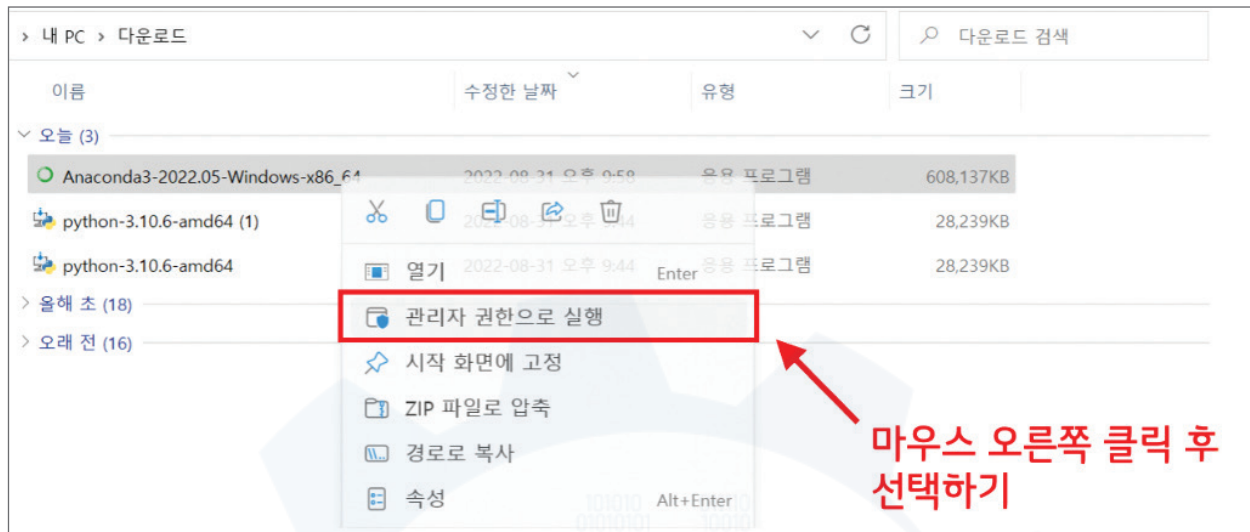
② 스크롤을 내려 다음의 화면을 확인한 후, 로컬 컴퓨터 사용 환경에 맞는 아나콘다 설치 파일을 다운로드한다.

- ▶ Windows : 64-Bit Graphical Installer
- ▶ MacOS : 64-Bit Graphical Installer

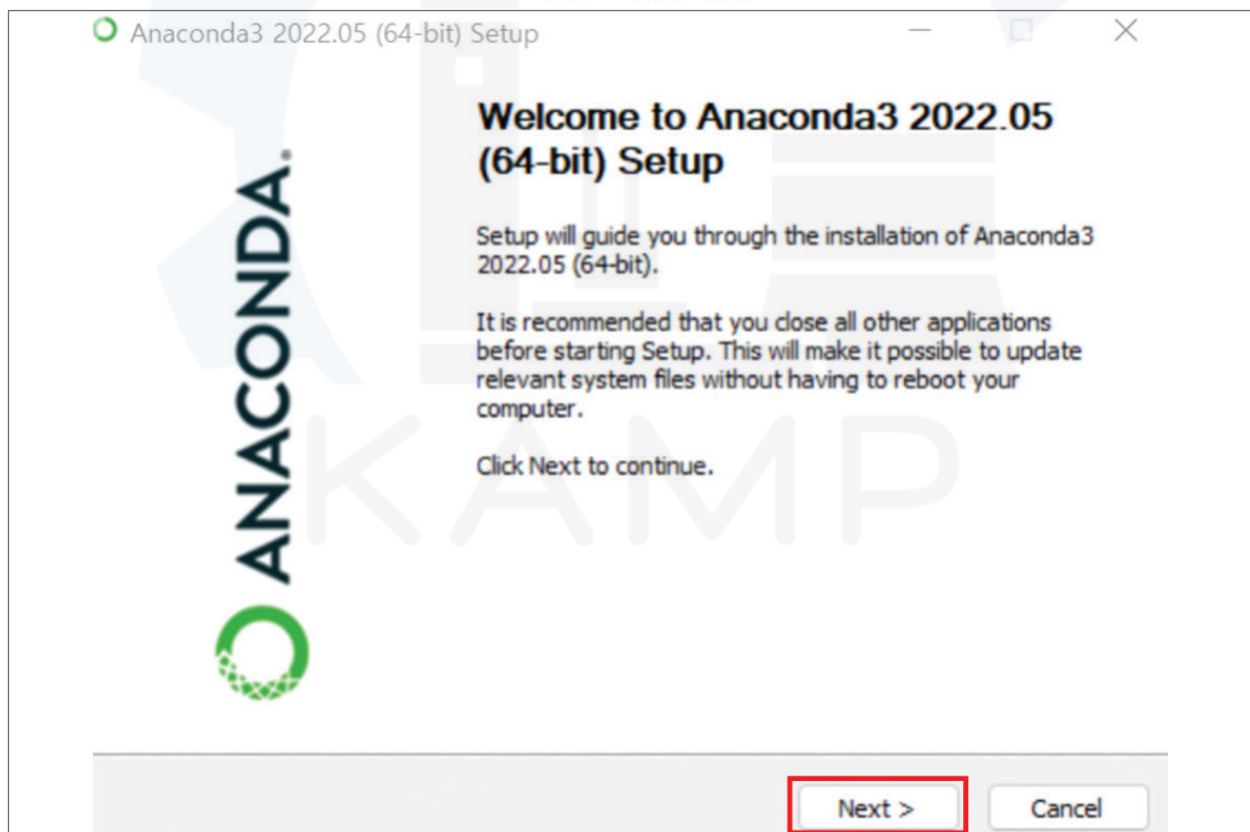


③ 다운로드한 아나콘다 설치 파일에서 마우스 오른쪽을 클릭한 후, 방패모양의 ‘관리자 권한으로 실행’을 클릭한다.

▶ (예) ‘다운로드’ 폴더로 아나콘다를 다운로드한 경우

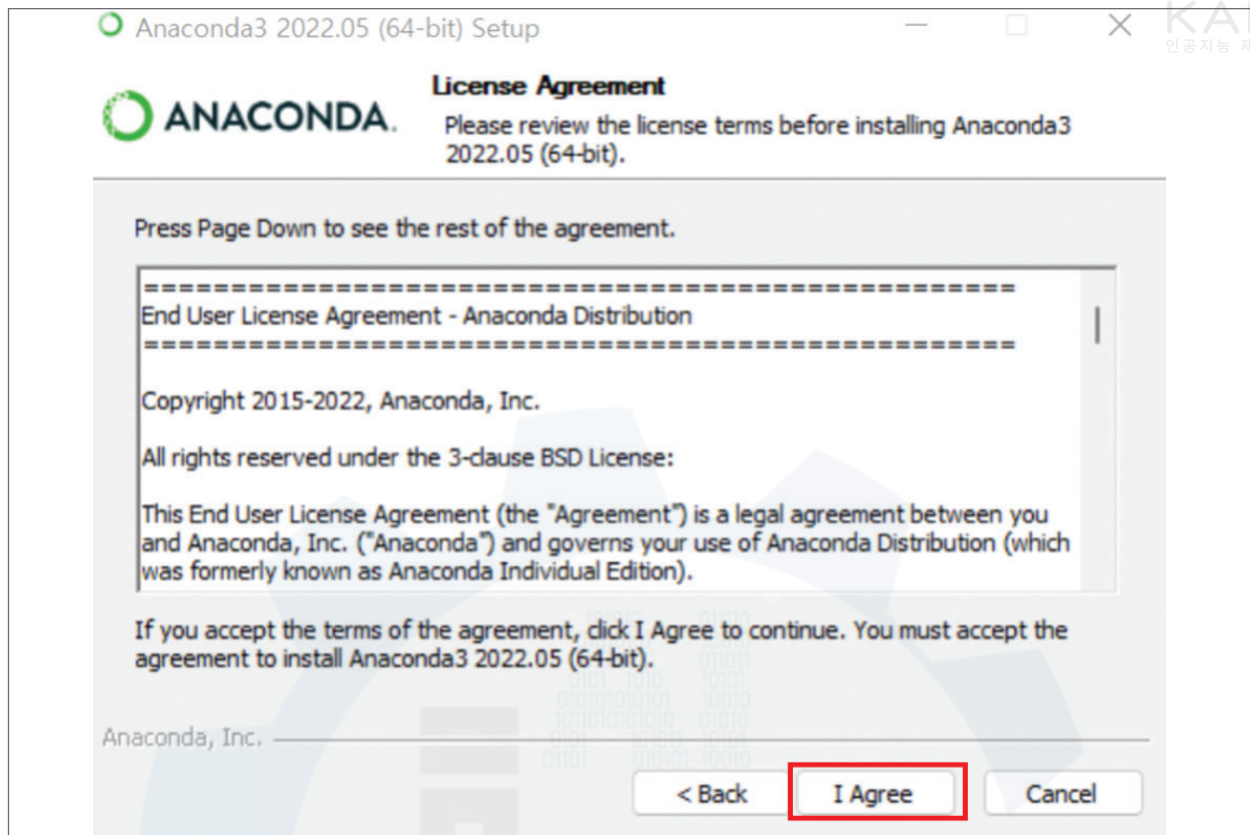


④ 설치 파일을 실행한 다음, ‘Next’ 버튼을 클릭한다.

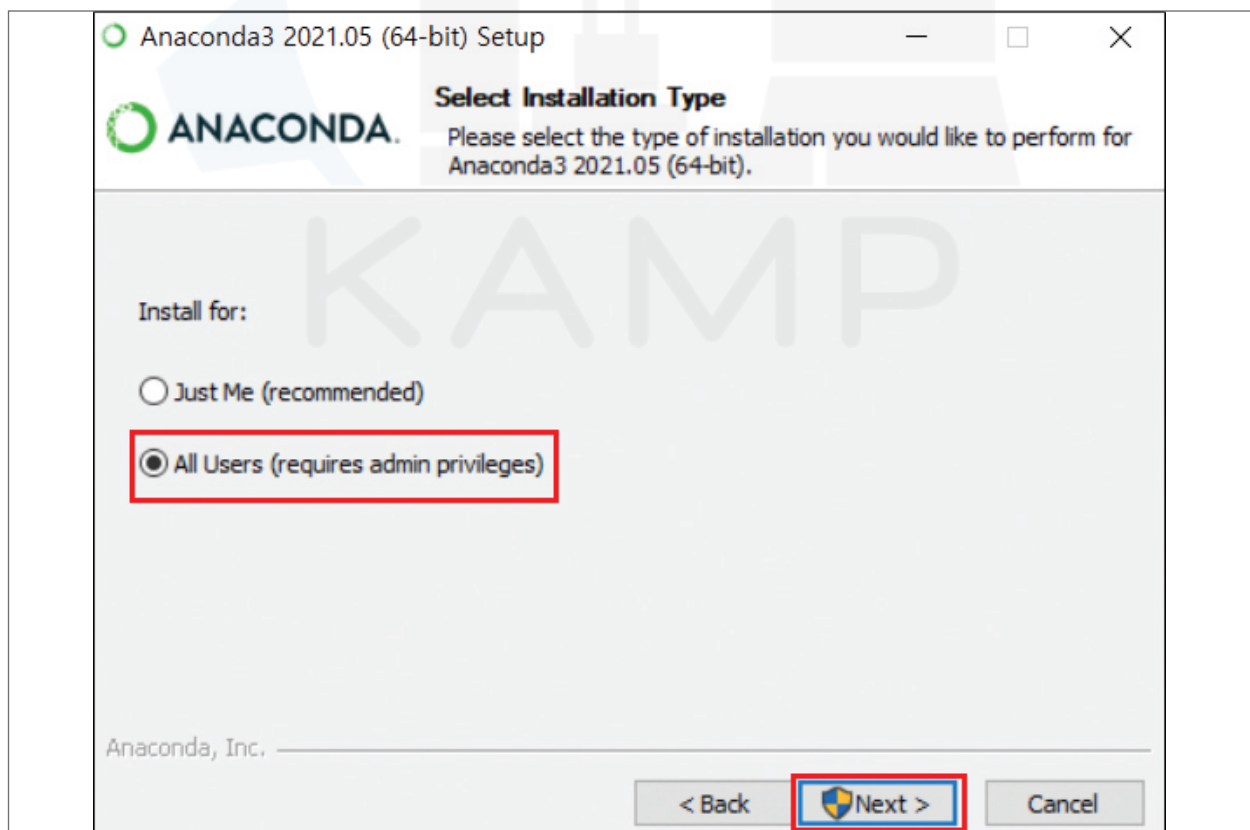




⑤ 다음 단계에서 아래와 같은 창이 나타나면, 'I Agree'를 선택한다.

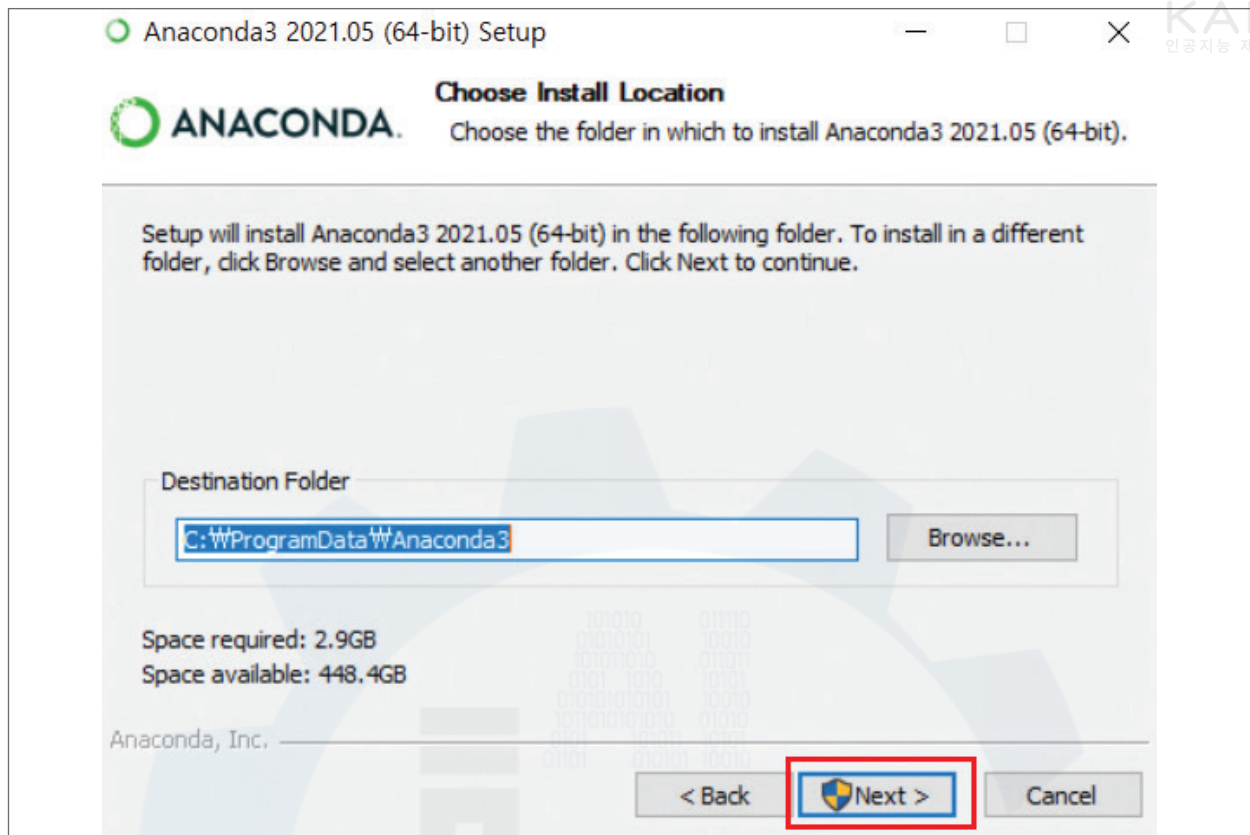


⑥ 아래와 같은 창이 나타나면, 'All Users'를 선택하여 다음 단계로 진행한다.

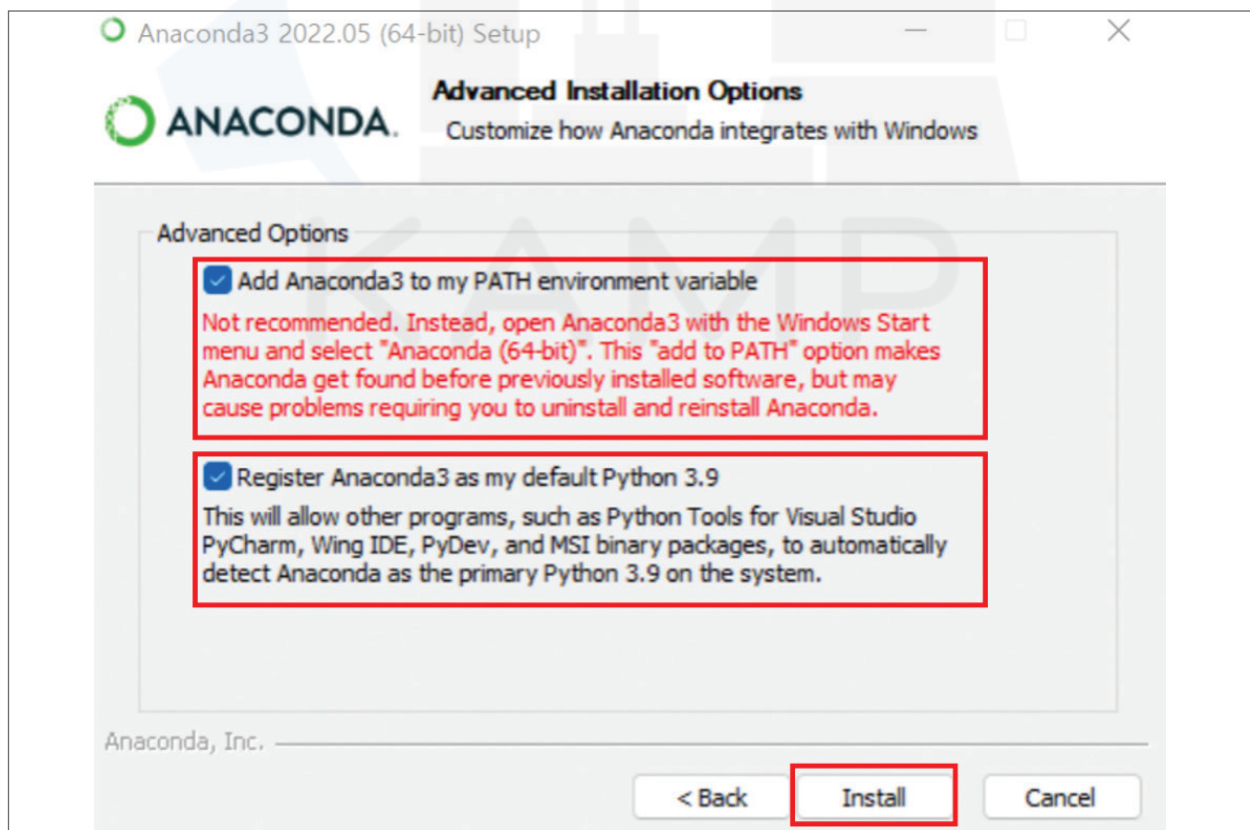




⑦ 다운로드할 경로를 설정한 뒤, 아래의 'Next'를 클릭한다.

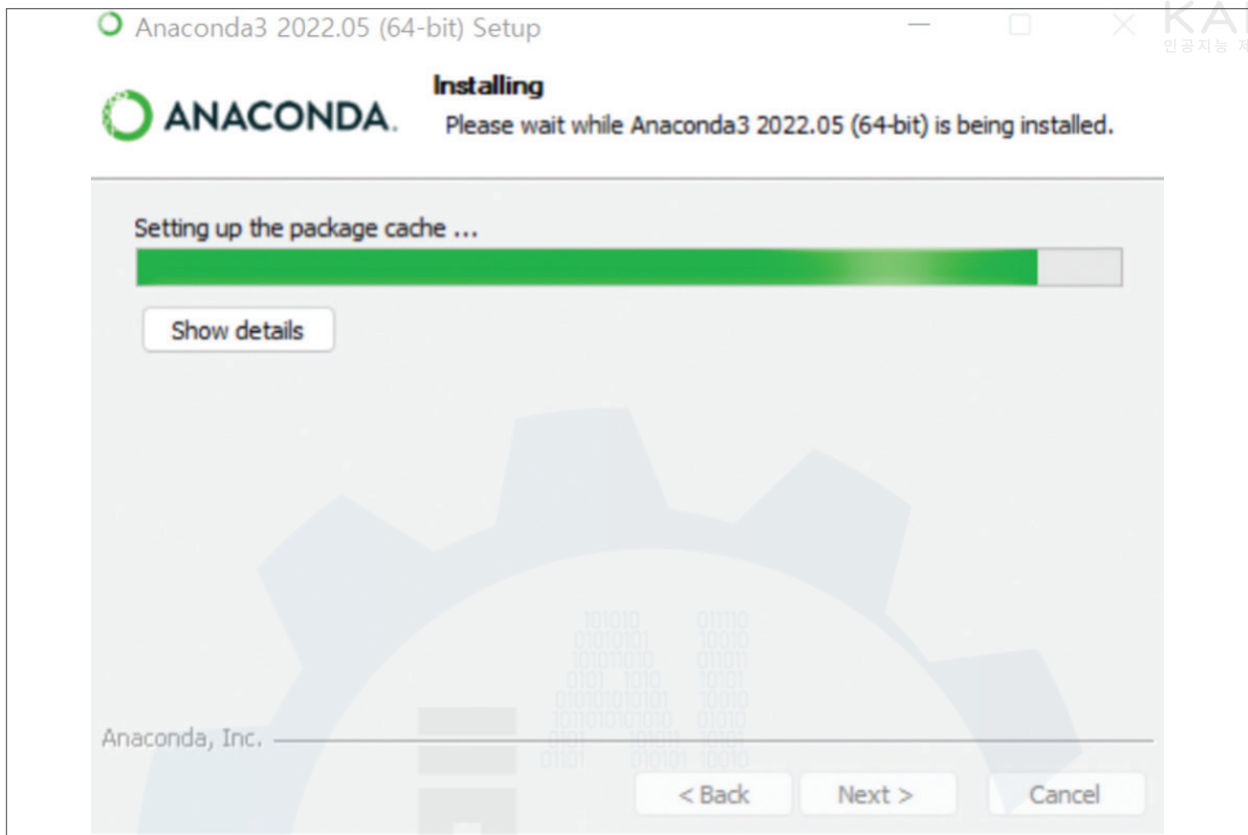


⑧ 고급 옵션 선택창에서 아래와 같이 모두 선택 후, 'Install'을 클릭한다.

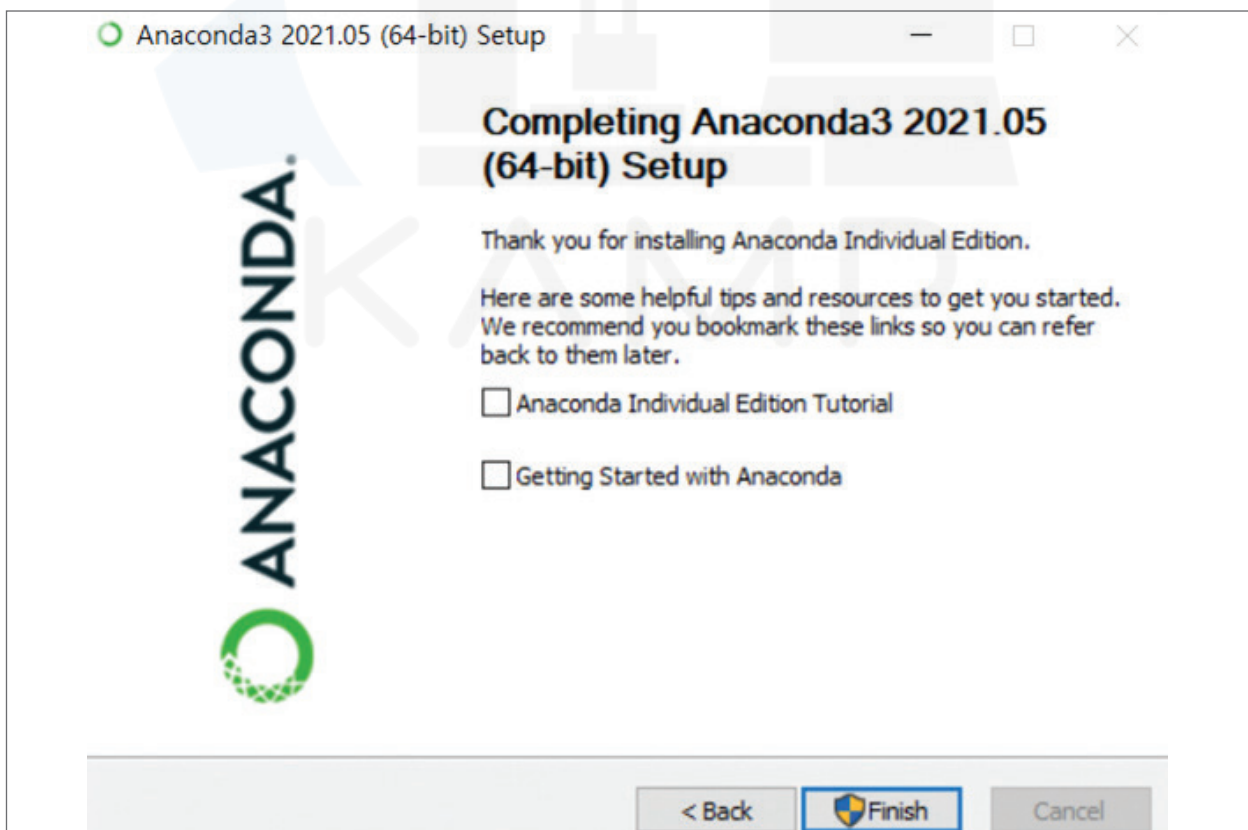




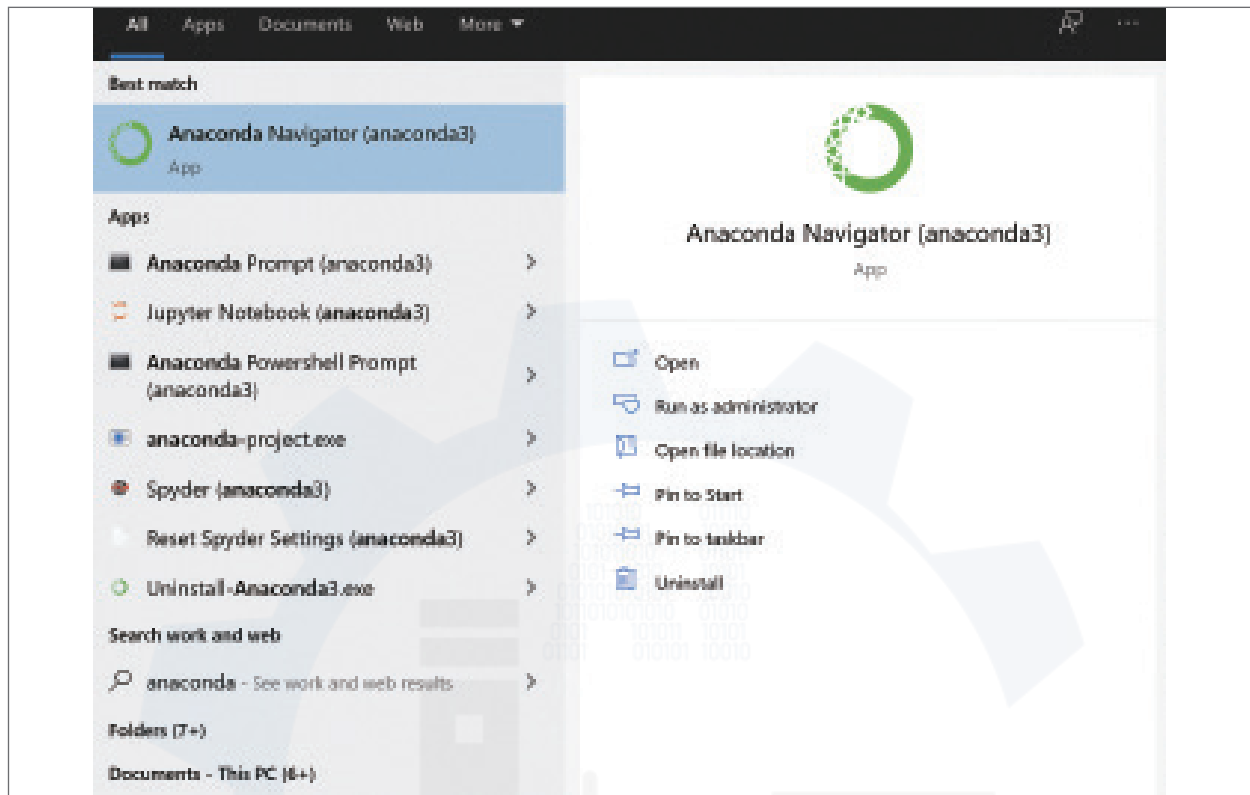
⑨ 다음과 같은 설치창으로 넘어가면, 완료가 될 때까지 대기한다. (5분이상 소요)



⑩ 마지막 화면에서 모두 체크 해제한 후, 'Finish'를 눌러 설치를 완료한다.



- ⑪ PC의 '시작 버튼(Windows)'을 눌러서 화면과 같이 anaconda prompt가 설치되었는지 확인한다. Anaconda Navigator, Anaconda Prompt, Jupyter Notebook 등의 다른 응용 프로그램들도 함께 확인이 된다면 설치가 완료된 것이다.

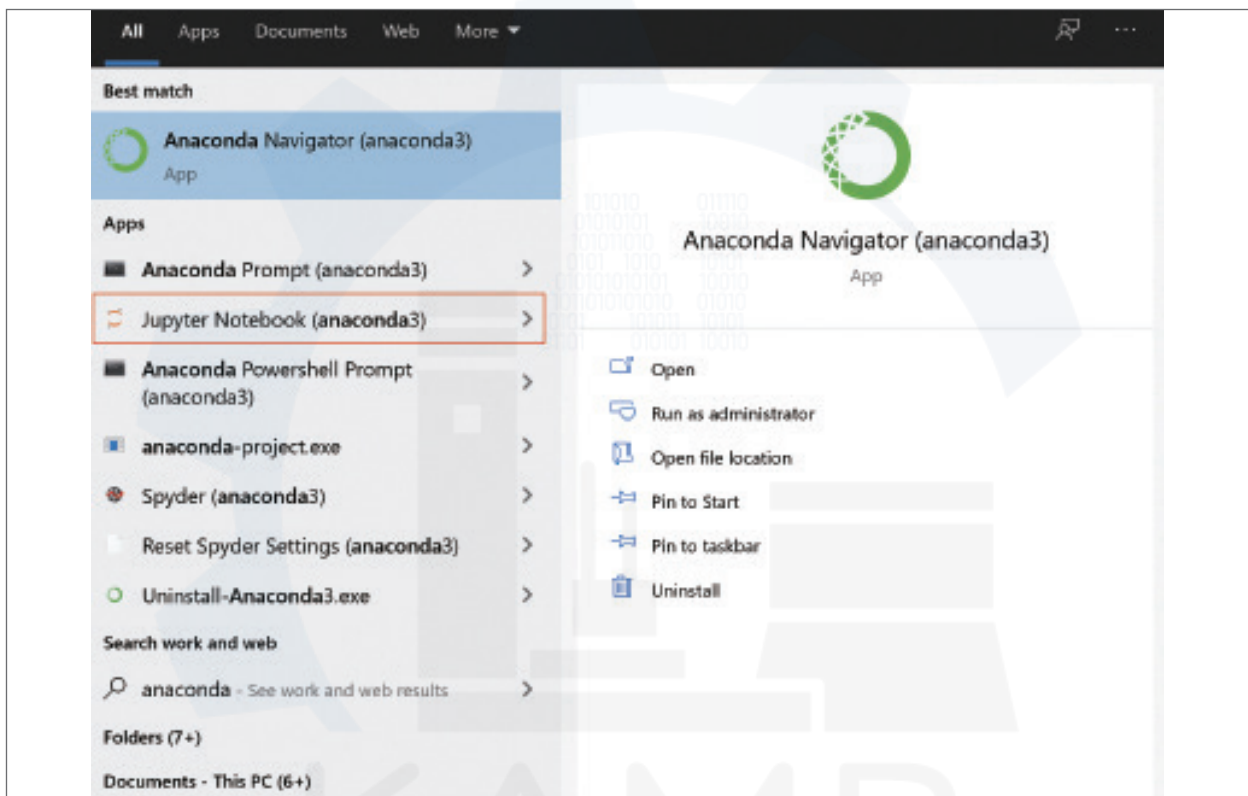




3. 주피터 노트북 (Jupyter Notebook) 실행

주피터 노트북은 사용자가 코딩(분석 코드 작성)을 할 수 있는 도구이다. 쉽게 비교하자면, 문서 도구로 마이크로소프트 사의 'Word' 프로그램이나, 한컴소프트 사의 '한글' 프로그램 등과 같은 도구라고 생각할 수 있다. 데이터 분석에 다양한 입력, 실행 도구가 있지만, 본 가이드북에서는 주피터 노트북을 활용하는 방법을 안내한다.

- ① PC의 '시작 버튼(Windows)'을 눌러서 화면과 같이 'anaconda'를 검색한 후 폴더 리스트 중 'Jupyter Notebook (anaconda3)'을 실행한다.





② Jupyter Notebook을 실행하면, 아래와 같이 2개의 윈도우가 실행된다.

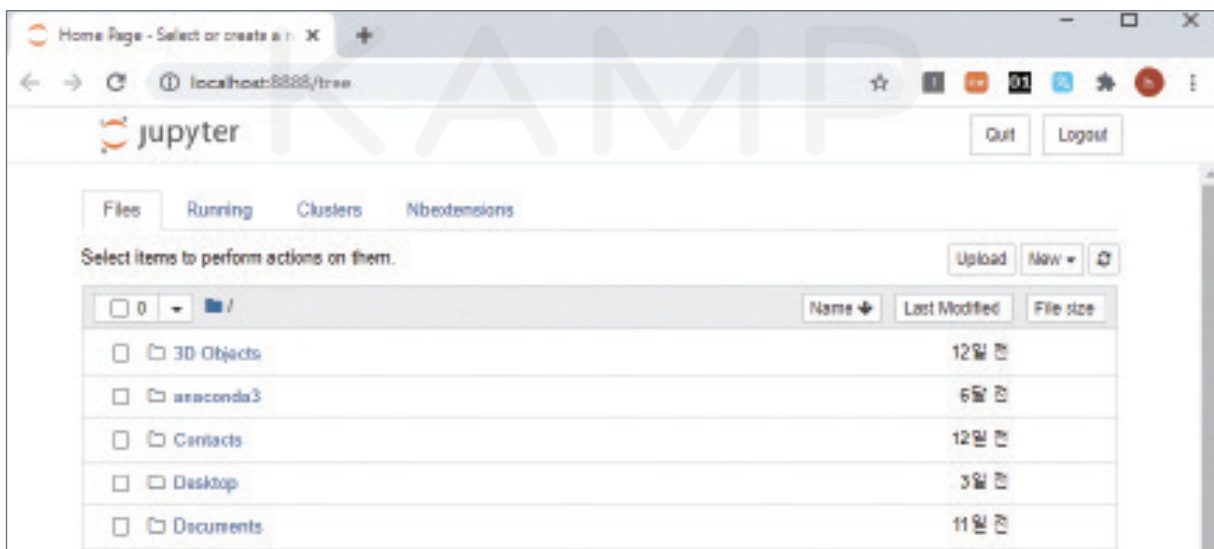
- (1) 아래 검은색 배경의 화면은 주피터 노트북이 실행되는 환경 상태를 나타내주는 상태 표시 창이다. 주피터 노트북을 사용하는 동안에는 종료하면 안된다.

```

Jupyter Notebook (anaconda3)
[I 12:39:38.566 NotebookApp] JupyterLab extension loaded from C:\Users\leedaeyoung\Anaconda3\lib\site-packages\jupyterlab
[I 12:39:38.566 NotebookApp] JupyterLab application directory is C:\Users\leedaeyoung\Anaconda3\share\jupyter\lab
[I 12:39:38.627 NotebookApp] Serving notebooks from local directory: C:\Users\leedaeyoung
[I 12:39:38.628 NotebookApp] The Jupyter Notebook is running at:
[I 12:39:38.628 NotebookApp] http://localhost:8888/?token=e6a3b83d88decc6e5b457c8fdb8b7b86d823ba8bc31a88c2
[I 12:39:38.628 NotebookApp] or http://127.0.0.1:8888/?token=e6a3b83d88decc6e5b457c8fdb8b7b86d823ba8bc31a88c2
[I 12:39:38.628 NotebookApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
[C 12:39:39.053 NotebookApp]

To access the notebook, open this file in a browser:
file:///C:/Users/leedaeyoung/AppData/Roaming/jupyter/runtime/nbserver-6568-open.html
Or copy and paste one of these URLs:
http://localhost:8888/?token=e6a3b83d88decc6e5b457c8fdb8b7b86d823ba8bc31a88c2
or http://127.0.0.1:8888/?token=e6a3b83d88decc6e5b457c8fdb8b7b86d823ba8bc31a88c2
  
```

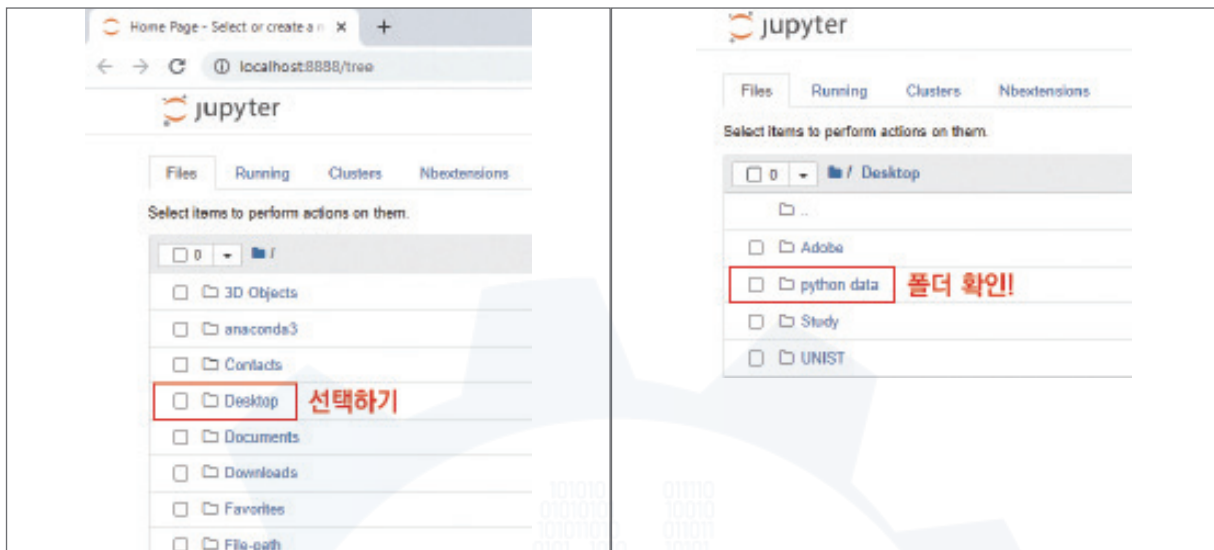
- (2) 기본 브라우저(크롬, 엣지 등)에서 주피터 노트북이 열린다. (다른 확장 프로그램 사용하고 싶다면 기본 브라우저를 변경해주어야 한다.)



③ 데이터 분석을 진행할 폴더를 생성한다.

▶ (예) '바탕화면'에 'python data' 폴더를 생성한 다음, 파이썬 코드 실행하고 저장한다.

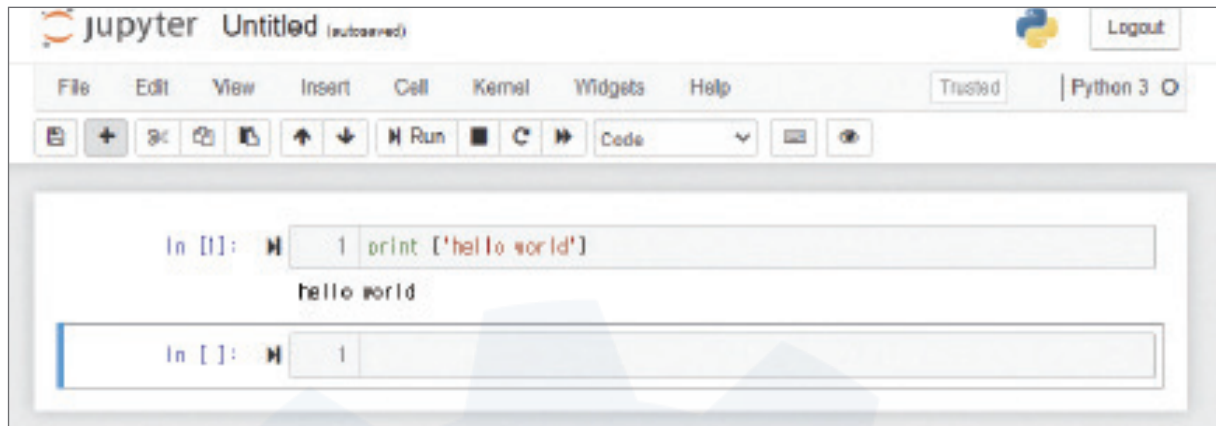
(1) 주피터 노트북에서 'python data' 폴더를 확인한다.



(2) python data에서 파이썬 파일을 생성한다. 오른쪽 상단의 'New'를 누른 후, 'Python 3'을 선택한다.



- (3) 'hello world' 출력을 확인해본다. 보이는 In [] 우측 회색 창에 print('hello world')를 입력한 다음, 상단의  Run 버튼 혹은 shift + Enter 키를 눌러서 실행한다. 코드 파일의 확장자는 '*.ipynb'로 저장된다.



4

부록_데이터 품질 전처리 [실습 코드]

KAMP
인공지능 제조 플랫폼

앞선 '2.분석 실습'에서 데이터 품질 전처리에 필요한, Gartner에서 제시한 6종의 데이터 품질 평가지표(완전성, 유일성, 유효성, 일관성, 정확성, 무결성)에 대하여 논의하였다.

```
import os
import pandas as pd
import datetime
df = pd.read_csv('일무사_표면처리.csv', encoding='utf-8')
df
```

	생산일자	작업지시번호	제품코드	시작	종료	생산량(Kg)	1차 출고날짜	2차 출고날짜	3차 출고날짜	4차 출고날짜
0	2021-01-12	2101130065	M21	2021-01-13 7:10	2021-01-13 10:40	13,550	2021-01-25	2021-01-26	2021-01-19	2021-01-28
1	2021-01-12	2101130066	M21	2021-01-13 10:40	2021-01-13 14:05	14,000	2021-01-19	2021-01-25	NaN	NaN
2	2021-01-13	2101130067	M21	2021-01-13 14:05	2021-01-13 17:30	14,000	2021-01-28	2021-02-09	2021-02-17	2021-02-03
3	2021-01-13	2101130068	M21	2021-01-13 17:30	2021-01-13 20:40	14,000	2021-01-26	2021-01-25	NaN	NaN
4	2021-01-13	2101140075	M21	2021-01-13 20:40	2021-01-13 23:50	15,500	2021-02-17	2021-02-25	2021-02-22	2021-03-12
...	...	...	...	...	...	...	...	...	...	...
172	2022-04-25	2204250797	M21	2022-04-25 13:30	2022-04-25 17:05	14,000	2022-05-13	2022-05-18	2022-05-02	NaN
173	2022-04-25	2204250798	M21	2022-04-25 17:05	2022-04-25 20:19	15,850	2022-05-06	2022-05-11	2022-05-16	NaN
174	2022-05-03	2205040846	M21	2022-05-04 4:30	2022-05-04 7:39	14,150	2022-05-16	2022-05-19	NaN	NaN
175	2022-05-03	2205040847	M21	2022-05-04 7:39	2022-05-04 10:54	14,000	NaN	NaN	NaN	NaN
176	2022-05-03	2205040848	M21	2022-05-04 10:54	2022-05-04 13:57	14,500	2022-05-18	2022-05-19	NaN	NaN

[데이터 준비]

▶ 완전성 품질 지수

$$\text{완전성 품질 지수} = \left(1 - \frac{\text{결측 데이터의 개수}}{\text{전체 데이터의 개수}}\right) \times 100$$

결측 데이터의 값이 30% 이상인 데이터들은 데이터의 완전성을 떨어뜨리기 때문에 각 열(column)에 대한 결측 데이터값의 비율을 확인하여 삭제한다.



```
perc = 30
df.isnull().sum()/len(df)*100
```

```
생산일자      0.000000
작업지시번호   0.000000
제품코드      0.000000
시작          0.000000
종료          0.000000
생산량(Kg)     0.000000
1차 출고날짜   0.564972
2차 출고날짜   2.824859
3차 출고날짜  25.423729
4차 출고날짜  56.497175
5차 출고날짜  75.706215
6차 출고날짜  84.745763
7차 출고날짜  92.655367
8차 출고날짜  94.915254
9차 출고날짜  97.740113
10차 출고날짜  98.870056
1차 출고량     0.000000
2차 출고량     0.000000
3차 출고량     0.000000
4차 출고량     0.000000
5차 출고량     0.000000
6차 출고량     0.000000
7차 출고량     0.000000
8차 출고량     0.000000
9차 출고량     0.000000
10차 출고량    0.000000
dtype: float64
```

```
perc = 30
df.isnull().sum()/len(df)*100 > perc
```

```
생산일자      False
작업지시번호   False
제품코드      False
시작          False
종료          False
생산량(Kg)     False
1차 출고날짜   False
2차 출고날짜   False
3차 출고날짜   False
4차 출고날짜   True
5차 출고날짜   True
6차 출고날짜   True
7차 출고날짜   True
8차 출고날짜   True
9차 출고날짜   True
10차 출고날짜  True
1차 출고량     False
2차 출고량     False
3차 출고량     False
4차 출고량     False
5차 출고량     False
6차 출고량     False
7차 출고량     False
8차 출고량     False
9차 출고량     False
10차 출고량    False
dtype: bool
```

[분석 도구를 활용한 완전성 품질지수 확인]

총 결측 데이터의 개수를 확인하기 위해 'isnull()' 함수를 사용하여 각 열(column)에서의 결측 데이터 개수를 구한 뒤 'sum()' 함수를 이용하여 합산한다.

```
cmpt_len = df.isnull().sum().sum()
print("결측치 = %d개 \n완전성 지수 : %.2f%%"%(cmpt_len,(1 - cmpt_len / df.count().sum())*100))
```

```
결측치 = 1115개
완전성 지수 : 68.02%
```

[분석 도구를 활용한 완전성 품질지수 구하기 1]

결측된 데이터들은 실제로 0 값이므로 값을 0으로 채운 후 그 결과를 확인한다.

```
df = df.fillna(0)
cmpt_len = df.isnull().sum().sum()
print("결측치 = %d개 \n완전성 지수 : %.2f%%"%(cmpt_len,(1 - cmpt_len / df.count().sum())*100))
```

```
결측치 = 0개
완전성 지수 : 100.00%
```

[분석 도구를 활용한 완전성 품질지수 구하기 2]

▶ 유일성 품질 지수

$$\text{유일성 품질 지수} = \left(\frac{\text{유일한 데이터의 개수}}{\text{전체 데이터의 개수}} \right) \times 100$$

데이터에서 유일한 값을 갖는 열(column)을 찾는다. (여기서는 time column)

만일 유일한 데이터 값을 갖는 열(column)이 존재하지 않는다면, 몇 개의 열(변수)을 결합한 데이터를 생성하여 유일한 데이터셋을 만들 수 있다.

유일한 값을 갖는 열(column) 작업지시번호에 대해 'groupby()' 함수를 이용하여, 각 열(column)에서의 유일한 데이터 값을 확인하고, 'size()' 함수를 이용하여 그 개수를 구한다.

```
df.groupby(['작업지시번호']).size()
```

```
작업지시번호
2101130065    1
2101130066    1
2101130067    1
2101130068    1
2101140075    1
..
2204250797    1
2204250798    1
2205040846    1
2205040847    1
2205040848    1
Length: 177, dtype: int64
```

[분석 도구를 활용한 유일한 값 확인하기]

유일한 데이터 개수를 이용하여 아래와 같이 유일성 품질지수를 구한다.

```
unq = len(df.groupby(['작업지시번호']).size())
persnt = (unq/len(df))*100
print("유일성 지수 : %.2f%%" % (persnt))
```

```
유일성 지수 : 100.00%
```

[분석 도구를 활용한 유일성 품질지수 확인하기]

▶ 유효성 품질지수

$$\text{유효성 품질 지수} = \left(\frac{\text{유효성을 만족한 데이터의 개수}}{\text{전체 데이터의 개수}} \right) \times 100$$

① 데이터가 유효한 범위에 있는가?

: 데이터셋에 정의된 수집 범위를 이용하여 조건(유효 범위)을 모두 충족하는지 확인한다.

```
cnt = 0
for c in df.columns:
    print(df[c].dtype)
```

[illegible]

[분석 도구를 활용한 유효성 품질지수 구하기(데이터 범위)-1]

현재 전처리 되지 않은 데이터이기 때문에 대부분이 object로 표기되는 것을 확인할 수 있다. 약간의 수정이 필요하다.

숫자 사이에 존재하는 콤마(,) 때문에 문자(object)로 인식하는 문제가 발생하여 콤마를 제거하고 다시 작업해보고자 한다.



```
cols = ['생산량(Kg)', '1차 출고량', '2차 출고량', '3차 출고량', '4차 출고량', '5차 출고량',
        '6차 출고량', '7차 출고량', '8차 출고량', '9차 출고량', '10차 출고량']
for c in cols:
    df[c] = df[c].apply(lambda x:int(x.replace(',','')))
for c in df.columns:
    print(df[c].dtype)
```

```
object
int64
object
object
object
int64
object
object
object
object
object
object
object
object
object
object
int64
int64
int64
int64
int64
int64
int64
int64
int64
int64
int64
```

[분석 도구를 활용한 유효성 품질지수 구하기(데이터 범위)-2]



자, 이제 데이터가 유효한 범위 내에 있는지를 확인해보도록 한다.

다음의 코드로부터 min과 max 값을 확인할 수 있는데, 이 값을 충족하는지 확인한 후 이 범위를 위반하는 데이터가 존재할 시에 경고문을 출력하고 값을 확인한다.

```
cnt = 0
for c in df.columns:
    if df[c].dtype != object:
        print(c+"의 min 값:", df[c].min(), c+"의 max 값:", df[c].max())
        c_min = df[c].min()
        c_max = df[c].max()
        tmp_chk = False
        for dc in df[c]:
            tmp_chk = (c_min <= dc)
            if not tmp_chk:
                print("non validation data detected", dc)
                cnt += 1
                break
        else: print(f'{c}:{df[c].dtype}')
val_1 = (1-cnt/len(df))
```

```
생산일자:object
작업지시번호의 min 값: 2101130065 작업지시번호의 max 값: 2205040848
제품코드:object
시작:object
종료:object
생산량(Kg)의 min 값: 7500 생산량(Kg)의 max 값: 17250
1차 출고날짜:object
2차 출고날짜:object
3차 출고날짜:object
4차 출고날짜:object
5차 출고날짜:object
6차 출고날짜:object
7차 출고날짜:object
8차 출고날짜:object
9차 출고날짜:object
10차 출고날짜:object
1차 출고량의 min 값: 0 1차 출고량의 max 값: 14000
2차 출고량의 min 값: 0 2차 출고량의 max 값: 13000
3차 출고량의 min 값: 0 3차 출고량의 max 값: 12000
4차 출고량의 min 값: 0 4차 출고량의 max 값: 10000
5차 출고량의 min 값: 0 5차 출고량의 max 값: 9000
6차 출고량의 min 값: 0 6차 출고량의 max 값: 6000
7차 출고량의 min 값: 0 7차 출고량의 max 값: 3500
8차 출고량의 min 값: 0 8차 출고량의 max 값: 2000
9차 출고량의 min 값: 0 9차 출고량의 max 값: 2000
10차 출고량의 min 값: 0 10차 출고량의 max 값: 1000
1.0
```

[분석 도구를 활용한 유효성 품질지수 구하기(데이터 범위)-2]

② 데이터가 형식에 맞는가?

데이터의 형식을 확인한다.

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 177 entries, 0 to 176
Data columns (total 26 columns):
#   Column                Non-Null Count  Dtype
---  -
0   생산일자              177 non-null    object
1   작업지시번호          177 non-null    int64
2   제품코드              177 non-null    object
3   시작                  177 non-null    object
4   종료                  177 non-null    object
5   생산량(Kg)            177 non-null    int64
6   1차 출고날짜          177 non-null    object
7   2차 출고날짜          177 non-null    object
8   3차 출고날짜          177 non-null    object
9   4차 출고날짜          177 non-null    object
10  5차 출고날짜          177 non-null    object
11  6차 출고날짜          177 non-null    object
12  7차 출고날짜          177 non-null    object
13  8차 출고날짜          177 non-null    object
14  9차 출고날짜          177 non-null    object
15  10차 출고날짜         177 non-null    object
16  1차 출고량            177 non-null    int64
17  2차 출고량            177 non-null    int64
18  3차 출고량            177 non-null    int64
19  4차 출고량            177 non-null    int64
20  5차 출고량            177 non-null    int64
21  6차 출고량            177 non-null    int64
22  7차 출고량            177 non-null    int64
23  8차 출고량            177 non-null    int64
24  9차 출고량            177 non-null    int64
25  10차 출고량           177 non-null    int64
dtypes: int64(12), object(14)
memory usage: 36.1+ KB
```

[분석 도구를 활용한 유효성 품질지수 구하기(데이터 형식)]

③ 다음의 코드를 실행하여, 2)의 data type을 이용하여 유효성 품질지수를 확인한다.

```
cnt = 0
for c in df.columns:
    tmp_chk = False
    col_c = ""
    for i, dc in enumerate(df[c]):
        if i == 0:
            if 'int' in str(type(dc)):
                col_c = 'int'
            elif 'float' in str(type(dc)):
                col_c = 'float'
            tmp_chk = (col_c in str(type(dc)))
        if not tmp_chk:
            cnt += 1
val_2 = (1 - cnt / len(df))
```

[분석 도구를 활용한 유효성 품질지수 구하기(데이터 형식 2)]

- ① 의 코드에서 구한 val_1과 data type을 이용하여 ③의 코드에서 구한 지수 val_2를 이용하여 아래와 같이 유효성 품질 지수를 구한다.

```
print("유효성 지수 : %.2f%%" % ((val_1+val_2)/2*100))
```

```
유효성 지수 : 72.32%
```

[분석 도구를 활용한 유효성 품질지수 구하기]

▶ 일관성 품질지수

$$\text{일관성 품질 지수} = \left(\frac{\text{일관성을 만족한 데이터의 개수}}{\text{전체 데이터의 개수}} \right) \times 100$$

본 데이터의 구조, 값, 형태가 일관이 있는지 확인한다. 'dtypes()' 함수를 이용하여 전체 데이터의 형태를 확인한다.

```
from collections import Counter
counter = Counter(df.dtypes.values)
counter
```

```
Counter({dtype('O'): 14, dtype('int64'): 12})
```

```
obj = counter.most_common()[0][1]
nt = counter.most_common()[1][1]
val = round(nt/(obj+nt)*100,3)
print("일관성 지수 : %.2f%%" % (val))
```

```
일관성 지수 : 46.15%
```

[분석 도구를 활용한 데이터 일관성 확인]

▶ 정확성 품질 지수

$$\text{정확성 품질 지수} = \left(\frac{\text{정확성에 위배되는 데이터의 개수}}{\text{전체 데이터의 개수}} \right) \times 100$$

- ① 데이터의 '평균값'이 수집된 데이터에 한하여, 다음과 같이 정확성 품질지수를 확인할 수 있다.
- ② 누락된 데이터가 있는지 확인하기 위해, '*.mean()' 함수를 이용하여 데이터들의 평균을 구한다.
- ③ '*.mean()' 함수로 구한 데이터의 평균에서 데이터의 '평균값'과의 차이를 구한다.
- ④ 이 차이가 0이 아니면 데이터를 위배하는 데이터가 존재하게 된다.

- 모든 열값이 독립적이므로 정확성 지수를 확인하지 않는다.



▶ 무결성 품질 지수

$$\text{무결성 품질지수} = \left\{ 1 - \left(\frac{\text{유일성, 유효성, 일관성 중 100\%가 아닌 지수}}{3} \right) \right\} \times 100$$

데이터의 유일성 100%, 유효성 72.31% 그리고 일관성은 46.15%이다. 따라서 무결성 품질 지수는 33.33%이다.

```
Integrity_result = f'{(1-(2/3))*100}%'
print(f'무결성 품질 지수:{Integrity_result} \n유일성 {persnt}%, 유효성 {round((val_1+val_2)/2*100,2)}%, 일관성 {round(val,2)}%')
```

무결성 품질 지수:33.333333333333336%
유일성 100.0%, 유효성 72.32%, 일관성 46.15%

[분석 도구를 활용한 무결성 품질지수 구하기]

▶ 가중치 지수

$$\text{가중치지수} = \text{품질지수} \times \text{가중치}$$

데이터의 특성에 따라 품질지수 항목의 중요도는 달라진다.

활용되는 데이터에서 중요한 품질지수가 무엇인지를 평가하고, 데이터의 특성에 따라 적절한 가중치를 부여한다.

▶ 데이터 품질 지수

$$\text{데이터품질지수} = \sum \text{가중치지수}$$

최종적으로 구한 6가지 데이터 품질지수에 적절한 가중치를 부여하여 품질지수를 구한다.

구분	품질지수	가중치	가중치 지수	오류율
완전성 (누락)	100%	10%	10%	0%
유일성 (중복)	100%	20%	20%	0%
유효성 (유효)	72%	50%	36%	14%
일관성 (표현)	46%	10%	5%	5%
정확성 (실제)	-	-	-	-
무결성	33%	10%	3%	7%
품질지수		100%	74%	26%



「표면처리 공급망최적화 AI 데이터셋」 분석실습 가이드북